Frozen CLIP-DINO: A Strong Backbone for Weakly Supervised Semantic Segmentation

Bingfeng Zhang, *Member, IEEE*, Siyue Yu, *Member, IEEE*, Jimin Xiao, *Member, IEEE*, Yunchao Wei, *Member, IEEE*, and Yao Zhao, *Fellow, IEEE*

***Abstract*—Weakly supervised semantic segmentation has witnessed great achievements with image-level labels. Several recent approaches use the CLIP model to generate pseudo labels for training an individual segmentation model, while there is no attempt to apply the CLIP model as the backbone to directly segment objects with image-level labels. In this paper, we propose WeCLIP and its advanced version WeCLIP+, to build the single-stage pipeline for weakly supervised semantic segmentation. For WeCLIP, the frozen CLIP model is applied as the backbone for semantic feature extraction, and a new light decoder is designed to interpret extracted semantic features for final prediction. Meanwhile, we utilize the above frozen backbone to generate pseudo labels for training the decoder. Such labels are fixed during training. We then propose a refinement module (RFM) to optimize them dynamically. For WeCLIP+, we introduce the frozen DINO model to achieve more comprehensive semantic feature extraction. The frozen DINO is combined with the frozen CLIP as the backbone, followed by a shared decoder to make predictions with less training cost. Moreover, a strengthened refinement module (RFM+) is designed to revise online pseudo labels with extra guidance from DINO features. Extensive experiments show that both WeCLIP and WeCLIP+ significantly outperform other approaches with less training cost. Particularly, WeCLIP+ gets mIoU of 83.9% on VOC 2012 *test* set and 56.3% on COCO *val* set. Additionally, these two approaches also obtain promising results for fully supervised settings.**

***Index Terms*—Weakly supervised, semantic segmentation, CLIP, DINO.**

I. Introduction

WEAKLY supervised semantic segmentation (WSSS) [1], [2], [3] aims to learn a pixel-level segmentation model from weak supervision so as to reduce the manual annotation efforts. The common weak supervision signals contain scribble [4], bounding-box [5], point [6] and image-level labels [7], [8], [9], [10]. Among these supervisions, image-level annotation is the most popular one, as such annotations can be easily obtained through web-crawling.

There are two training solutions for WSSS with image-level labels: multi-stage training and single-stage training. For existing single-stage approaches, their backbones rely on pre-training on ImageNet [11] and fine-tuning during training, as in Fig. 1(a). Such single-stage training [12], [13] focuses on using one model to directly segment objects with weak signals as supervision. The primary consideration of previous single-stage architectures is to online refine the Class Activation Map (CAM) [14] or to improve the segmentation branch [15], [16]. Due to the complicated architecture, single-stage approaches perform normally worse than multi-stage approaches.

On the other hand, multi-stage training attempts to utilize several individual models to form a training pipeline [8], [17], [18], where offline pixel-level pseudo labels are first generated from weak labels using CAM [19] and then a segmentation model is trained with such pseudo labels. Since CAM can only highlight discriminate regions, many previous approaches focus on improving the quality of CAM [20], [21], [22], [23] for better pseudo labels. Besides, some recent multi-stage approaches [24], [25], [26] attempt to introduce Contrastive Language-Image Pre-training (CLIP) [27] for WSSS. Trained on 400 million image-text pairs, CLIP establishes a strong relationship between the image and text, demonstrating great ability to locate objects [24], [28], [29], [30], [31]. Based on this, existing approaches [24], [26] use CLIP to improve CAM, providing surprisingly high-quality pseudo labels. They follow the pipeline in Fig. 1(b). However, these methods only use the CLIP model to improve CAM for better pseudo labels. The potential of the CLIP model to be directly used as the backbone to extract strong semantic features for segmentation prediction is not explored.

To address the above limitations, we propose to directly use the frozen CLIP model as the backbone for semantic feature extraction. First, we design **WeCLIP** [32], as demonstrated in Fig. 1(c), which adopts the frozen CLIP model as the backbone, followed by a newly designed light frozen CLIP feature decoder, where the CLIP backbone does not need any training or fine-tuning. Our decoder can successfully interpret the frozen CLIP features to conduct the segmentation task with

Received 7 August 2024; revised 15 December 2024; accepted 5 February 2025. Date of publication 18 February 2025; date of current version 3 April 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 62301613, Grant 62301451, Grant 62471405, and Grant 62331003, in part by the Taishan Scholar Program of Shandong under Grant tsqn202306130, in part by the Suzhou Basic Research Program under Grant SYG202316, in part by Basic Research Program of Jiangsu under Grant SBK2024021981, in part by Shandong Natural Science Foundation under Grant ZR2023QF046, in part by Qingdao Postdoctoral Applied Research Project under Grant QDBSH20230102091, in part by the Independent Innovation Research Project of UPC under Grant 22CX06060A, and in part by the XJTLU Research Development Fund under Grant RDF-22-02-066. Recommended for acceptance by J. Dai. *(Corresponding authors: Siyue Yu; Jimin Xiao.)*

Bingfeng Zhang is with the China University of Petroleum (East China), Qingdao 257099, China (e-mail: bingfeng.zhang@upc.edu.cn).

Siyue Yu and Jimin Xiao are with the School of Advanced Technology, Xi'an Jiaotong-Liverpool University, Suzhou 215000, China (e-mail: siyue.yu02@xjtlu.edu.cn; jimin.xiao@xjtlu.edu.cn).

Yunchao Wei and Yao Zhao are with the Institute of Information Science, Beijing Jiaotong University, Beijing 100044, China (e-mail: yunchao.wei@bjtu.edu.cn; yzhao@bjtu.edu.cn).

The code is available at https://github.com/zbf1991/WeCLIP.

Digital Object Identifier 10.1109/TPAMI.2025.3543191

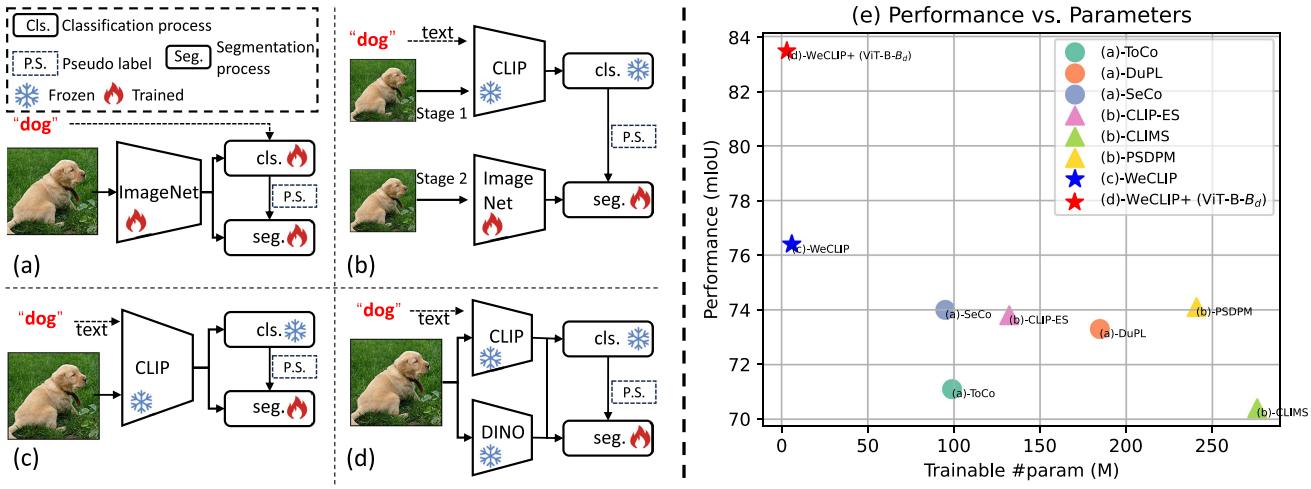


Fig. 1. Comparisons between our approach and other single-stage or CLIP-based approaches. (a) Previous single-stage approach, which uses a trainable ImageNet [11] pre-trained backbone with trainable classification and segmentation process. (b) Previous CLIP-based approach, which is a multi-stage approach that uses the Frozen CLIP model to produce pseudo labels and trains an individual ImageNet pre-trained segmentation model. (c) Our proposed WeCLIP, which is a single-stage approach that uses a frozen CLIP model as the backbone with a trainable segmentation process, significantly reducing the training cost. (d) Our proposed WeCLIP+, which combines the Frozen CLIP and DINO to build a new single-stage approach. (e) Performance comparison between different methods on PASCAL VOC 2012 *val* set, both WeCLIP and WeCLIP+ have fewer learnable parameters but higher performance.

a small number of learnable parameters. Considering that we utilized the frozen CLIP backbone to generate CAMs for providing pixel-level pseudo labels to train our decoder, the pseudo labels are static during training. The same errors in pseudo labels lead to uncorrectable optimization in the wrong directions. Thus, we further propose a frozen CLIP CAM **ReF**inement **M**odule (RFM) to rectify the static CAM dynamically. Particularly, our RFM utilized the dynamic features from our decoder and the prior features from the frozen CLIP backbone to establish high-quality pair-wise feature relationships to revise the initial CAM, leading to higher-quality pseudo labels.

However, the CLIP model is an image-language model that aligns the global image-level feature with the corresponding text feature. It is hard to ensure that detailed semantic information is extracted from the image. WeCLIP has a pure CLIP-based backbone, which heavily relies on its frozen features. Once the representation of the features is not accurate, the following learning process will not be guaranteed.

Accordingly, we design an advanced version, **WeCLIP+**, by combining the frozen DINO [33] and frozen CLIP to build a stronger backbone for weakly supervised semantic segmentation, as shown in Fig. 1(d). Compared to WeCLIP, WeCLIP+ has two main improvements: First, it is proved that the DINO [33] model trained through self-supervised learning can provide explicit semantic information [34], [35], [36] without any manual annotations. Thus, the introduced frozen DINO replenishes the frozen CLIP for more detailed semantic representations. WeCLIP+ only requires features of the last layer from both the frozen CLIP and the frozen DINO models for decoding. In this way, WeCLIP+ significantly decreases the number of learnable parameters to 58% of WeCLIP and 24% of previous methods. Second, we re-design a frozen CLIP-DINO CAM **ReF**inement **M**odule (RFM+) by decoding the combination of CLIP and DINO features to re-weight the static CAMs, providing more reliable online pseudo labels than RFM. With such designs, on the one hand, the frozen CLIP is complemented by the frozen DINO to build a strengthened backbone with fewer learnable parameters for the decoder. Fig. 1(e) shows that both WeCLIP and WeCLIP+ have fewer learnable parameters but higher performance than other frameworks, and the performance of WeCLIP+ outperforms WeCLIP by a clear margin. On the other hand, our proposed two modules can also benefit from each other: refined pseudo labels provide more accurate supervision to train the decoder, and the trained decoder builds more reliable feature relationships for RFM+ to generate accurate pseudo labels.

Extensive experiments show that our approach achieves new state-of-the-art performances on both the PASCAL VOC 2012 and MS COCO datasets and significantly outperforms other approaches by a large margin. Further, our approach also achieves satisfactory performance for fully supervised semantic segmentation. Our contributions are summarized as:

- We build powerful single-stage pipelines, WeCLIP, and the advanced version WeCLIP+, for weakly supervised semantic segmentation without fine-tuning the pre-trained model, significantly reducing the training costs. WeCLIP thoroughly interprets multi-layer frozen CLIP features to segment objects with our designed light decoder. Our WeCLIP+ uses the frozen DINO to replenish the frozen CLIP feature, building a strengthened single-stage solution.
- With the complementary DINO features, our CLIP-DINO backbone can extract rich and highly compact semantic information, thus WeCLIP+ only requires features from the last layers of frozen CLIP and DINO models for decoding, strikingly decreasing the number of learnable parameters to 58% of WeCLIP.

- To overcome the drawback that the frozen backbone only provides static pseudo labels, we design a frozen CLIP-DINO CAM refinement module (RFM+) to dynamically update the initial CAM, providing continuously improved supervision to train our model.
- With less training cost, our approach significantly outperforms previous approaches, reaching a new state-of-the-art performance for weakly supervised semantic segmentation (mIoU: 83.9% on VOC 2012 *test* set, 56.3% on COCO *val* set). Moreover, our approach also shows great potential for fully supervised semantic segmentation.

II. Related Work

A. Weakly Supervised Semantic Segmentation

Weakly supervised semantic segmentation with image-level supervision [7], [12], [24], [37] attracts more attention than other weak supervisions [4], [5] due to less human effort. There are two main solutions: multi-stage approaches [1], [3], [10], [18], [25] and single-stage approaches [12], [14].

The key to the multi-stage solution is to generate high-quality pseudo labels. For example, RIB [18] designed a margin loss in the classification network to reduce the information bottleneck, producing better pixel-level responses from image-level supervision. Du et al. [22] proposed a pixel-to-prototype contrast strategy to impose feature semantic consistency to generate higher-quality pseudo labels. MCTformer [9] designed multi-class tokens in the transformer architecture to produce class-specific attention responses to generate refined CAM. However, AReAM [38] discovered that over-smoothing in deep transformer layers would cause background noise in CAMs. Thus, AReAM supervised high-level attention with shallow affinity matrices, ensuring that the high-level attention can focus on semantic relationships. Some recent multi-stage approaches attempted to introduce CLIP for this task. CLIMS [24] utilized the CLIP model to activate more complete object regions and suppress highly related background regions. CLIP-ES [26] proposed to use the softmax function in CLIP to compute the GradCAM [39]. With carefully designed text prompts, the GradCAM of CLIP provided reliable pseudo labels to train the segmentation model. Based on CLIP-ES [26], PSDAM [40] proposed a Prototype-based Secondary Discriminative Pixels Mining framework to mine more secondary discriminative pixels, thus providing high-quality pseudo labels. Meanwhile, CPAL [41] focused on enhancing the prototype by mitigating the bias to capture better spatial semantic relationships, which also aims to provide high-quality pseudo labels. Additionally, WeakCLIP [42] proposed to use learnable text prompts to obtain better text-image matching for high-quality pseudo labels. Besides, S2C [43] introduced Segment Anything Model (SAM) [44], a powerful foundational model for prompt-based segmentation, to transfer its knowledge to the classifier for generating better pseudo labels. In addition to generating high-quality pseudo labels, another solution is to extract reliable information from the pseudo labels for segmentation training. For instance, OCR [45] adopted a group ranking mechanism to refine the out-of-candidate pixels as proper semantic pixels.

Previous single-stage solutions adopted the ImageNet [11] pre-train model as the backbone to concurrently learn the classification and segmentation tasks, and most of them focused on improving segmentation by providing more accurate supervision or constraining its learning. For example, RRM [12] proposed to select reliable pixels as supervision for the segmentation branch. 1Stage [13] designed a local consistency refinement module to directly generate semantic masks from image-level labels. AA&AR [16] proposed an adaptive affinity loss to enhance semantic propagation in the segmentation branch. ASDT [46] introduced a dual-teacher single-student network architecture to mine full object regions and discriminative object features for more reliable online pseudo labels. SLRNet [47] aggregates multi-view features of the input to help the network learn more precise online pseudo labels. Besides, AFA [14] designed an affinity branch to refine CAMs to generate better online pseudo labels. ToCo [48] proposed token contrast learning to mitigate over-smoothing in online CAM generation, thus providing better supervision for segmentation. FSR [49] proposed a progressive feature self-reinforcement method to strengthen the semantics of uncertain regions, especially for object boundaries and misclassified categories, to improve the final performance. DuPL [50] proposed a dual student framework to build a single-stage solution, which reduced the over-activation rate of online CAM and improved its quality efficiently. SeCO [51] designed a separate and conquer strategy to handle the frequent co-occurring objects problem with a dual-teacher-single-student architecture.

The CLIP model shows great effectiveness in the multi-stage solution, but using it as a single-stage solution, i.e., directly learning to segment objects with image-level supervision, is not explored.

B. Fully Supervised Semantic Segmentation

Fully supervised semantic segmentation aims to segment objects using pixel-level labels as supervision. Most previous approaches are based on Fully Convolutional Network (FCN) [52] architecture, such as DeepLab [53], PSPNet [54], and UperNet [55]. Based on these fundamental methods, IDRNet [56] investigated co-occurrent visual patterns to aggregate more meaningful contextual information. Contextrast [57] proposed contextual contrastive learning to help the network grasp both local and global semantic feature representation. On the other hand, recent approaches introduced vision transformer [58] as the backbone to improve performance by building global relationships. For example, PVT [59] used a pyramid vision transformer for semantic segmentation. Swin [60] designed a window-based attention mechanism in the vision transformer to effectively improve attention computing. They added a UperNet head [55] for semantic segmentation. MaskFormer [61] and Mask2Former [62] proposed universal image segmentation architecture by combining the transformer decoder and pixel decoder. No matter whether fully or weakly supervised semantic segmentation, almost all segmentation models rely on the ImageNet [11] Pre-train models, and all the model parameters need to train or finetune, which requires a large number of computing costs, while we use the frozen models e.g., the frozen CLIP and

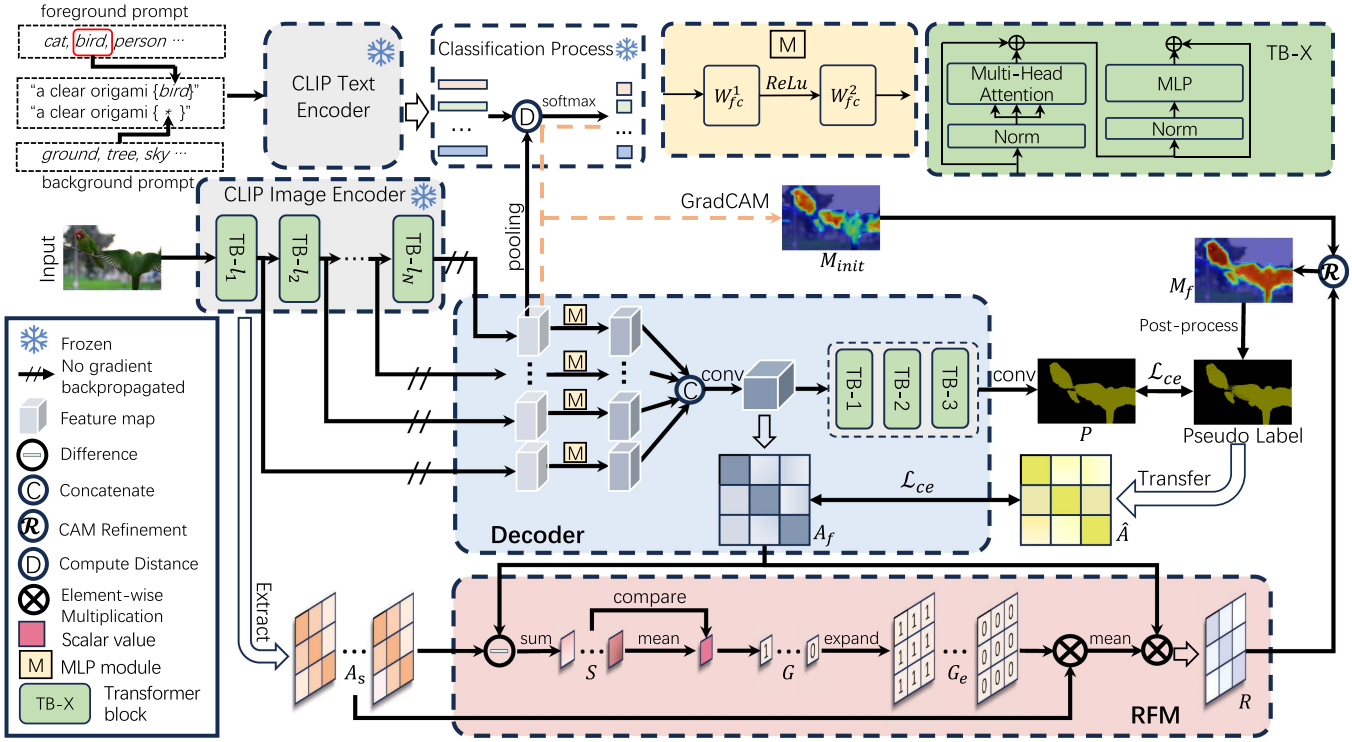


Fig. 2. Framework of our WeCLIP. The image is input to the Frozen CLIP image encoder to generate the image features, and class labels are used to build text prompts and then input to the Frozen CLIP text encoder to generate the text features. The classification scores are generated based on the distance between the pooled image and text features. Using GradCAM, we can generate the initial CAM M_{init}. Then, the frozen image features from the last layer of each transformer block are input to our decoder to generate the final semantic segmentation predictions. Meanwhile, the affinity map A_f from our decoder and the multi-head attention maps A_s from CLIP are input to our RFM to establish refining maps R to refine M_{init} as M_f. After post-processing, it will be used as the supervision to train the decoder.

frozen DINO models, as the backbone, leading to much less resource on the computation.

III. Method

A. WeCLIP

1) Overview of WeCLIP: Fig. 2 shows the whole framework of WeCLIP, including four main modules: a frozen CLIP backbone (image encoder and text encoder) to encode the image and text, a classification process to produce initial CAM, a decoder to generate segmentation predictions, a RFM to refine initial CAM to provide pseudo labels for training.

The training pipeline is divided into the following steps:

1) First of all, the image is input to the CLIP image encoder for image features. Besides, the foreground and background class labels are used to build text prompts and then input to the CLIP text encoder to generate the corresponding text features. Note here both image and text encoders are frozen during training.
2) Then, the classification scores are generated by computing distances between image features (after pooling) and text features. Based on classification scores, GradCAM [39] is utilized to generate the initial CAM.
3) Besides, image features from the last layer of each transformer block in the frozen CLIP image encoder are input to our proposed decoder for the final segmentation predictions.
4) Simultaneously, the intermediate feature maps from our decoder are used to generate an affinity map. Then, the affinity map is input to our proposed RFM with the multi-head attention maps from each block of the frozen CLIP image encoder.
5) Finally, RFM outputs a refining map to refine the initial CAM. After post-processing, the final converted pseudo label from refined CAM is used to supervise the training.

2) Frozen CLIP Feature Decoder: We use the frozen CLIP encoder with ViT-B as the backbone, which is not optimized during training. Therefore, how to design a decoder that interprets CLIP features to semantic features becomes a core challenge. We propose a light decoder based on the transformer architecture to conduct semantic segmentation using CLIP features as input.

Specifically, suppose the input image is $I \in \mathbb{R}^{3\times H\times W}$, H and W represent the height and width of the image, respectively. After passing the CLIP image encoder, we generate the initial feature maps $\{F^l_{\text{init}}\}^N_{l=1}$ from the output of each transformer block in the encoder, where l represents the index of the block. Then, for each feature map F^l_{init}, an individual MLP module is used to generate new corresponding feature maps F^l_{new}:

$$F^l_{\text{new}} = W^1_{\text{fc}}\left(\text{ReLU}\left(W^2_{\text{fc}}\left(F^l_{\text{init}}\right)\right)\right), \tag{1}$$

where W_{fc}^1 and W_{fc}^2 are two different fully-connected layers. ReLU$(\cdot)$ is the ReLU activation function.

After that, all new feature maps $\{F_{\text{new}}^l\}_{l=1}^N$ are concatenated together, which are then processed by a convolution layer to generate a fused feature map F_u:

$$F_u = \text{Conv}\left(\text{Concat}\left[F_{\text{new}}^1, F_{\text{new}}^2, .., F_{\text{new}}^N\right]\right), \tag{2}$$

where $F_u \in \mathbb{R}^{d\times h\times w}$, where d, h, and w represent the channel dimension, height, and width of the feature map. Conv$(\cdot)$ is a convolutional layer, Concat$[\cdot]$ is the concatenation operation.

Finally, we design several sequential multi-head transformer layers to generate the final prediction P:

$$P = \text{Conv}\left(\phi\left(F_u\right)\right)\uparrow, \tag{3}$$

where $P \in \mathbb{R}^{C\times H\times W}$, C is the class number including background. $\phi(\cdot)$ represents the sequential multi-head transformer blocks [58], each block contains a multi-head self-attention module, a feed-forward network, and two normalization layers, as shown in the upper right corner of Fig. 2. $\uparrow$ is an upsample operation to align the prediction map size with the original image.

3) Frozen CLIP CAM Refinement: To provide supervision for the prediction P in (3), we generate the pixel-level pseudo label from the initial CAM of the frozen backbone. The frozen backbone can only provide static CAM, which means pseudo labels used as supervision cannot be improved during training. The same errors in pseudo labels lead to uncorrectable optimization in the wrong directions. Therefore, we design the frozen CLIP CAM **ReF**inement **M**odule (RFM) to dynamically update CAM to improve the quality of pseudo labels.

We first follow [26] to generate the initial CAM. For the given Image I with its class labels, I is input to the CLIP image encoder. The class labels are used to build text prompts and input to the CLIP text encoder. Then, the extracted image features (after pooling) and text features are used to compute the distance and further activated by the softmax function to get the classification scores. After that, we use GradCAM [39] to generate the initial CAM $M_{\text{init}} \in \mathbb{R}^{(|C_I|+1)\times h\times w}$, where $(|C_I|+1)$ indicates all class labels in the image I including the background. More details can be found in [26].

To thoroughly utilize the prior knowledge of CLIP, the CLIP model is fixed. Although we find that such a frozen backbone can provide strong semantic features for the initial CAM with only image-level labels, as illustrated in Fig. 3(a), M_{init} cannot be optimized as it is generated from the frozen backbone, limiting the quality of pseudo labels. Therefore, how to rectify M_{init} during training becomes a key issue. Our intuition is to use feature relationships to rectify the initial CAM. However, we cannot directly use the attention maps from the CLIP image encoder as the feature relationship, as such attention maps are also fixed. Nevertheless, the decoder is constantly being optimized, and we attempt to use its features to establish feature relationships to guide the selection of attention values from the CLIP image encoder, keeping useful prior CLIP knowledge and removing noisy relationships. With more reliable feature relationships, the CAM quality can be dynamically enhanced.

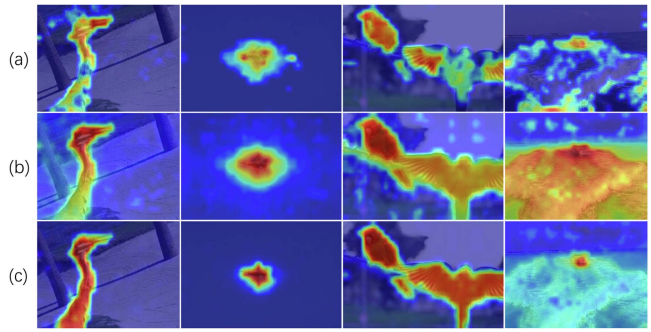


Fig. 3. Qualitative comparison of the CAM. (a) Initial CAM. (b) Refined CAM by attention maps proposed in [26]. (c) Our refined CAM. Our method produces more accurate responses.

In detail, we first generate an affinity map based on the feature map F_u in (2) from our decoder:

$$A_f = \text{Sigmoid}\left(F_u^T F_u\right), \tag{4}$$

where $F_u \in \mathbb{R}^{d\times h\times w}$ is first flattened to $\mathbb{R}^{d\times hw}$. Sigmoid$(\cdot)$ is the sigmoid function to guarantee the range of the output is from 0 to 1. $A_f \in \mathbb{R}^{hw\times hw}$ is the generated affinity map. T means matrix transpose.

Then we extract all the multi-head attention maps from the frozen CLIP image encoder, denoted as $\{A_s^l\}_{l=1}^N$ and each $A_s^l \in \mathbb{R}^{hw\times hw}$. For each A_s^l, we use A_f as a reference map to evaluate its quality:

$$S^l = \sum_{i=1}^{hw}\sum_{j=1}^{hw}\left|A_f(i,j) - A_s^l(i,j)\right|. \tag{5}$$

We use the above S^l to compute a filter for each attention map:

$$G^l = \begin{cases} 1, & \text{if } S^l < \frac{1}{N-N_0+1}\sum_{l=N_0}^N S^l \\ 0, & \text{else} \end{cases}, \tag{6}$$

where $G^l \in \mathbb{R}^{1\times 1}$, and it is expanded to $G_e^l \in \mathbb{R}^{hw\times hw}$ for further computation. We use the average value of all S^l as the threshold. If the current S^l is less than the threshold, it is more reliable, and we set its filter value as 1. Otherwise, we set the filter value as 0. Based on this rule, we keep high-quality attention maps and remove weak attention maps.

We then combine A_f and the above operation to build the refining map:

$$R = \frac{A_f}{N_m}\sum_{l=1}^N G_e^l A_s^l, \tag{7}$$

where N_m is the number of valid A_s^l, i.e., $N_m = \sum_{l=N_0}^N G^l$.

Then, following the previous approaches [26], we generate the refined CAM:

$$M_f^c = \left(\frac{R_{\text{nor}} + R_{\text{nor}}^T}{2}\right)^\alpha \cdot M_{\text{init}}^c, \tag{8}$$

where c is the specific class, M_f^c is the refined CAM for class c. R_{nor} is obtained from R using row and column normalization

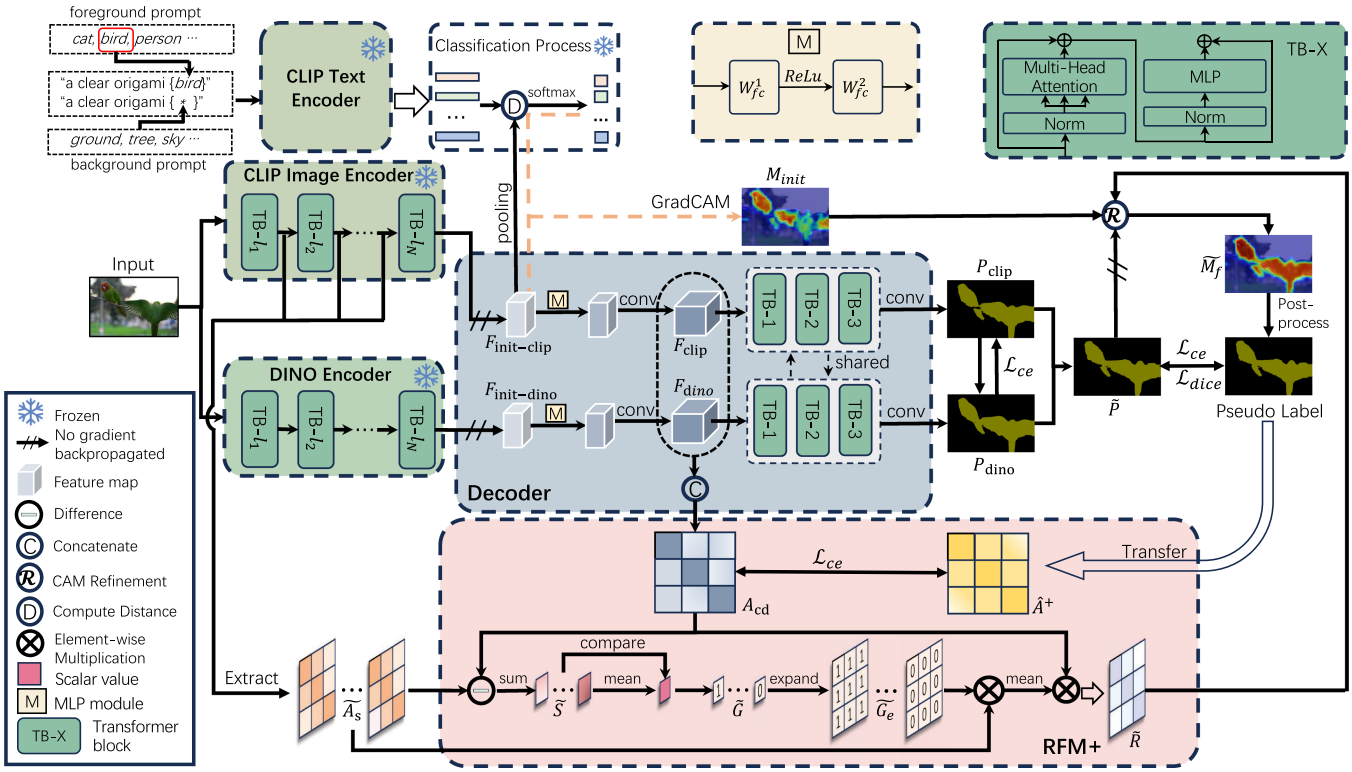


Fig. 4. The framework of our WeCLIP+. The image is input to the Frozen CLIP image encoder and Frozen DINO to generate two individual image feature maps, and class labels are used to build text prompts and then input to the Frozen CLIP text encoder to generate the text features. The classification scores are generated based on the distance between the pooled image and text features. Using GradCAM, we can generate the initial CAM M_{init}. Then, the frozen image feature maps are input to the shared decoder to generate two individual semantic segmentation predictions, P_{clip} and P_{dino}, respectively. After fusing P_{clip} and P_{dino}, the final prediction $\tilde{P}$ is generated. Meanwhile, the affinity map $\tilde{A}_f$ from our decoder and the multi-head attention maps $\tilde{A}_s$ from CLIP are input to our RFM+ to establish refining maps $\tilde{R}$, which further combine the final prediction $\tilde{P}$ to refine M_{init} as $\tilde{M}_f$. After post-processing, it will be used as the supervision to train the decoder.

(Sinkhorn normalization [63]). α is a hyper-parameter. This part passes a box mask indicator [26] to restrict the refining region. M_{init}^c is the CAM for class c after reshaping to $\mathbb{R}^{hw\times 1}$.

Finally, M_f is input to the online post-processing module, i.e., pixel adaptive refinement module proposed in [14], to generate final online pseudo labels $M_p \in \mathbb{R}^{h\times w}$.

In this way, our RFM uses the updated feature relationship in our decoder to assess the feature relationship in the frozen backbone to select reliable relationships. Then, higher-quality CAM can be generated with the help of more reliable feature relationships for each image. Fig. 3 shows the detailed comparison of generated CAM using different refinement methods. Our method generates more accurate responses than the static refinement method proposed in [26] and the initial CAM.

4) Loss Function: In our RFM, we use the affinity map A_f to select the attention map and build the final refining map. Therefore, the effectiveness of A_f directly determines the quality of the online pseudo labels. Considering A_f is generated using the feature map F_u in our decoder, and is a learnable module, we propose a learning process for A_f that uses the converted online pseudo label from M_p as supervision.

Specifically, M_p is first converted to the pixel-wise affinity label for each pair of pixels:

$$\hat{A} = O_h\left(M_p\right)^T O_h\left(M_p\right), \tag{9}$$

where $O_h(\cdot)$ is one-hot encoding and $O_h(M_p) \in \mathbb{R}^{C\times hw}$, $\hat{A} \in \mathbb{R}^{hw\times hw}$ is the affinity label. $\hat{A}(i,j) = 1$ means pixel i and j has the same label, otherwise, $\hat{A}(i,j) = 0$.

Based on the above label $\hat{A}$ and the online label M_p, the whole loss function of our WeCLIP is:

$$\mathcal{L} = \mathcal{L}_{ce}\left(P, M_p\uparrow\right) + \lambda \mathcal{L}_{ce}\left(A_f, \hat{A}\right), \tag{10}$$

where $\mathcal{L}_{ce}$ is the cross-entropy loss, $M_p\uparrow \in \mathbb{R}^{H\times W}$, and λ is the weighting parameter. P is the prediction in (3). With (10), more accurate feature relationships are established for higher-quality pseudo labels. In turn, with better pseudo labels, more precise feature relationships are established. Thus, our decoder and RFM can benefit from each other to boost the training.

B. WeCLIP+

1) Overview of WeCLIP+: Fig. 4 shows the whole framework of the WeCLIP+. Compared to WeCLIP, WeCLIP+ has several improvements: First, the backbone is changed from frozen CLIP to combined frozen CLIP and DINO, therefore, the image is encoded by both the CLIP image encoder and the DINO encoder. Second, instead of using features from all the last layers of each block in the CLIP, we only select the features from the last layer of the last block in the frozen CLIP and DINO as input for the decoder. Third, with the introduced DINO features, the

RFM is strengthened to RFM+ for better revising pseudo labels. All other training pipeline follows the same settings in WeCLIP, as described in Section III-A1.

2) Frozen CLIP-DINO Feature Decoder: We combine the frozen CLIP and frozen DINO as the backbone for WeCLIP+. Unlike WeCLIP, which uses feature maps from all the last layers of each transformer block, we only select the feature maps from the last layer of the frozen CLIP model and DINO model as input to the decoder since the feature map from DINO contains sufficient semantic information.

Specifically, given the original image $I \in \mathbb{R}^{3\times H\times W}$, since the frozen CLIP model and frozen DINO model might have different resolutions, e.g., the CLIP model is ViT-16 while the DINO model is ViT-14, the input image I is first resized into two images $I_{r_1} \in \mathbb{R}^{3\times h_1\times w_1}$ and $I_{r_2} \in \mathbb{R}^{3\times h_2\times w_2}$, respectively, where h_1 and h_2 are height, w_1 and w_2 are width to represent different resolutions. Then I_{r_1} and I_{r_2} are input to the frozen CLIP image encoder and the DINO model to generate two individual corresponding initial feature maps $F_{\text{init-clip}}$ and $F_{\text{init-dino}}$:

$$F_{\text{init-clip}} = E_{\text{clip}}(I_{r_1}), F_{\text{init-dino}} = E_{\text{dino}}(I_{r_2}), \tag{11}$$

where $E_{\text{clip}}(\cdot)$ and $E_{\text{dino}}(\cdot)$ represent the frozen CLIP image encoder and the frozen DINO encoder, respectively. Note that Both $F_{\text{init-clip}}$ and $F_{\text{init-dino}}$ are from the last layer of their corresponding encoders.

Then, both initial features are input to the MLP module and the convolution operations to generate two new feature maps:

$$\begin{aligned} F_{\text{clip}} &= \text{Conv}_1\left(\text{MLP}_1\left(F_{\text{init-clip}}\right)\right) \\ F_{\text{dino}} &= \text{Conv}_2\left(\text{MLP}_2\left(F_{\text{init-dino}}\right)\right)\downarrow, \end{aligned} \tag{12}$$

where $F_{\text{clip}} \in \mathbb{R}^{d\times h\times w}$ and $F_{\text{dino}} \in \mathbb{R}^{d\times h\times w}$. Besides, $\downarrow$ represents downsampling to ensure that F_{clip} and F_{dino} have the same size. MLP_1 and MLP_2 are two individual MLP modules, which share the same definition with (1), i.e., $\text{MLP}(\cdot) = W^1_{\text{fc}}(\text{ReLU}(W^2_{\text{fc}}(\cdot)))$. $\text{Conv}_1(\cdot)$ and $\text{Conv}_2(\cdot)$ are two individual convolution operations.

After that, F_{clip} and F_{dino} are input to the shared transformer layers and convolutional layer to generate their predictions:

$$\begin{aligned} P_{\text{clip}} &= \text{Conv}\left(\phi\left(F_{\text{clip}}\right)\right)\uparrow \\ P_{\text{dino}} &= \text{Conv}\left(\phi\left(F_{\text{dino}}\right)\right)\uparrow, \end{aligned} \tag{13}$$

where $P_{\text{clip}} \in \mathbb{R}^{C\times H\times W}$ and $P_{\text{dino}} \in \mathbb{R}^{C\times H\times W}$, ϕ, Conv and $\uparrow$ share the same definition with (3). Note that $\phi(\cdot)$ and $\text{Conv}(\cdot)$ are shared between P_{clip} and P_{dino}.

Finally, P_{clip} and P_{dino} are fused to generate the final prediction:

$$\widetilde{P} = \left(P_{\text{clip}} + P_{\text{dino}}\right)/2. \tag{14}$$

Compared to the original decoder in WeCLIP, the new decoder only uses two individual MLP modules and two individual convolution operations in (12) with the shared transformer blocks in (13). With such a design, our WeCLIP+ requires much fewer learnable parameters than WeCLIP. More importantly, we find that the shared transformer blocks can perform better predictions. This is because the shared transformer blocks can benefit from both CLIP features and DINO features (for more details, please see Table XIV). Besides, we do not introduce feature maps from different layers since the DINO feature can represent high-quality semantic information [34]. The feature maps of the last layer from both CLIP and DINO have contained sufficient information for further processing.

3) Frozen CLIP-DINO CAM Refinement: Similar to the RFM in Section III-A3, we hope to rectify the fixed CAM dynamically through the learnable feature maps. With the new proposed decoder that introduces the DINO feature, we design a strengthened RFM, called frozen CLIP-DINO CAM **ReF**inement **M**odule (RFM+), to achieve better online CAM refinement.

Specifically, we first concatenate two feature maps as follows:

$$F_{\text{cd}} = \text{Concat}\left(F_{\text{clip}}, F_{\text{dino}}\right), \tag{15}$$

where $F_{\text{cd}} \in \mathbb{R}^{2d\times h\times w}$, F_{clip} and F_{dino} are from (12). Then, we can generate a new affinity map:

$$A_{\text{cd}} = \text{Sigmoid}\left(F^T_{cd}F_{cd}\right), \tag{16}$$

Using A_{cd} as input, after passing through (5)-(7), we can generate the new refined map $\widetilde{R}$, and the refined CAM is defined as:

$$\widetilde{M}^c_f = \left(\frac{\widetilde{R}_{\text{nor}} + \widetilde{R}^T_{\text{nor}}}{2}\right)^{\alpha} \cdot \left(\widetilde{P}^c \odot M^c_{\text{init}}\right). \tag{17}$$

where $\widetilde{R}_{\text{nor}}$ is obtained from $\widetilde{R}$ using row and column normalization (Sinkhorn normalization [63]). $\odot$ means hadamard product. $\widetilde{P}$ is from (14) and the result of $\widetilde{P}^c \odot M^c_{\text{init}}$ is reshaped to $\mathbb{R}^{hw\times 1}$. Note that the gradient of $\widetilde{P}$ is cut off when used for this equation.

Since F_{cd} in (15) contains both CLIP feature and DINO feature, the corresponding affinity map A_{cd} in (16) contains stronger pair-wise relationships. Thus, the refining map $\widetilde{R}$ can provide more precise refining information for better CAM. In addition, we add term $\widetilde{P}$ in (17) as the predictions in WeCLIP+ are more accurate compared with WeCLIP, and with the help of $\widetilde{P}$, pixels with less probability can be suppressed to ensure a more confident refined CAM.

Finally, following the process in Section III-A3, $\widetilde{M}_f$ is input to the online post-processing module to generate final online pseudo labels $\widetilde{M}_p \in \mathbb{R}^{h\times w}$.

Compared to the original RFM in WeCLIP, RFM+ benefits from the strong semantic representations of the combined CLIP and DINO. RFM+ clearly promotes the quality of CAMs, providing more reliable online pseudo labels during training. Fig. 5 shows the detailed visualization of the online pseudo labels generated by WeCLIP and WeCLIP+, respectively. It can be observed that our WeCLIP+ can provide better online pseudo labels.

4) Loss Function: The whole loss function follows the setting in Section III-A4 and some changes are adopted to new designs in WeCLIP+:

$$\mathcal{L} = \mathcal{L}_{seg} + \lambda\mathcal{L}_{ce}\left(A_{\text{cd}}, \hat{A}^+\right), \tag{18}$$

where $\mathcal{L}_{ce}(\cdot)$ is the cross-entropy loss, $\hat{A}^+$ is generated by (9) by replacing M_p with $\widetilde{M}_p$. $\mathcal{L}_{seg}$ is the loss function of the decoder,

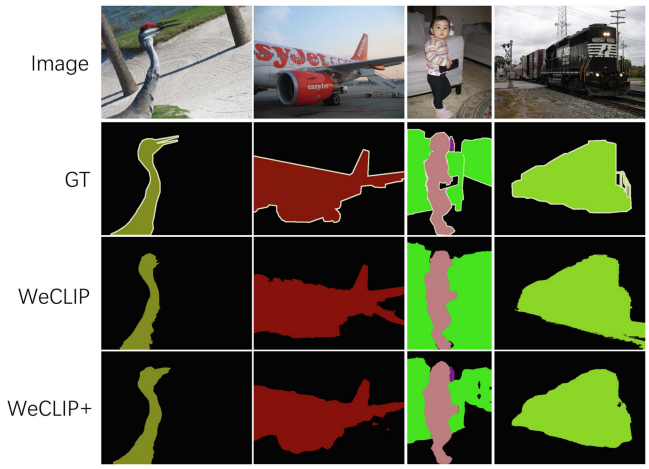


Fig. 5. Online pseudo label comparison between our WeCLIP and WeCLIP+. It can be found that online pseudo labels generated by WeCLIP+ are more complete and accurate.

defined as follows:

$$\mathcal{L}_{seg} = \mathcal{L}_{ce}\left(\widetilde{P}, \widetilde{M}_p \uparrow\right) + \mathcal{L}_{dice}\left(\widetilde{P}, \widetilde{M}_p \uparrow\right) + \beta\left(\mathcal{L}_{ce}(P_{\text{clip}}, M_{\text{dino}} \uparrow) + \mathcal{L}_{ce}\left(P_{\text{dino}} \uparrow, M_{\text{clip}}\right)\right). \tag{19}$$

In (19), $\mathcal{L}_{dice}$ is the Dice loss [64]. β is the weighting parameter. M_{dino} and M_{clip} represent the predicted pixel-level masks using the prediction P_{dino} and P_{clip}, respectively, i.e., $M_{\text{dino}} = \text{argmax}(P_{\text{dino}})$ and $M_{\text{clip}} = \text{argmax}(P_{\text{clip}})$. Compared to the loss function in the original WeCLIP, we add two new losses for detailed supervision: a new Dice loss to supervise the final prediction and two cross-entropy losses to cross-supervise the predictions based on the CLIP and DINO features, respectively. In light of the cross-supervision, the two branches can learn the information from each other to make better predictions.

IV. Experiment

A. Datasets

Following the setting in most previous weakly supervised semantic segmentation approaches [10], [18], [22], two datasets are used to evaluate our approach: PASCAL VOC 2012 [65] and MS COCO-2014 [66]. PASCAL VOC 2012 is appended with SBD [67] to expand the dataset, and the whole dataset contains 10,582 training images, 1,446 validation images, and 1,456 test images with 20 foreground classes. The MS COCO-2014 dataset includes approximately 82,000 training images and 40,504 validation images with 80 foreground classes.

Mean Intersection-over-Union (mIoU) is applied as the evaluation criterion.

B. Implementation Details

Pseudo Label Generation: we follow CLIP-ES [26] to define the foreground class set and background class set. Specifically, the background class set is ground, land, grass, tree, building, wall, sky, lake, water, river, sea, railway, railroad, keyboard, helmet, cloud, house, mountain, ocean, road, rock, street, valley, bridge, sign. Then, the text prompt is in the form of 'a clear origami {foreground class/background class},' which is illustrated in the upper left corner in Figs. 2 and 4.

WeCLIP: We use the frozen CLIP backbone with the ViT-16-base architecture [68] (ViT-B-16), N is a fixed number that equals 12. For training on the PASCAL VOC 2012 dataset, the batchsize is set as 4, and the maximum iteration is set as 30,000. For training on the MS COCO-2014 dataset, we set batchsize as 8, and the maximum iteration as 80,000. The learning rate is is $2e^{-3}$ with weight decay $1e^{-3}$.

All images are cropped to 320×320 during training. All other settings adopt the same parameters for two datasets during training: We use AdamW [69] as the optimizer, λ in (10) is set as 0.1, The dimension of the MLP module ((1)) in our decoder is set as 256. In ϕ of (3), three transformer encoder (the multi-head number is 8) layers are cascaded to generate the final feature map, and each layer's output dimension is 256. N_0 in (6) is set as 6. α is set as 2 in (8) following [26].

WeCLIP+: The frozen CLIP backbone is the ViT-B-16 architecture [68], DINOv2-ViT-S-14 and DINOv2-ViT-B-14 [33] are employed as the frozen DINO backbone. During training, all images are cropped to 320×320 for the frozen CLIP backbone. For the frozen DINO backbone, all images are cropped to 308×308. β in (19) is set as 0.1. All other settings are the same as the original WeCLIP.

To simplify the expression, in the following contexts, "VIT-B-S_d" is used to represent that the CLIP model is ViT-B-16 and the DINO model is ViT-S-14. "VIT-B-B_d" indicates that the CLIP model is ViT-B-16 and the DINO model is ViT-B-14.

During inference, we use the multi-scale with $\{0.75, 1.0\}$ for WeCLIP and $\{1.0, 1.5\}$ for WeCLIP+. Following previous approaches [14], [48], [81], DenseCRF [82] is used as the post-processing method to refine the prediction.

C. Comparison With State-of-the-Art Methods

In Table I, we compare our approach with other state-of-the-art approaches on the PASCAL VOC 2012 dataset. It can be seen that our WeCLIP+ reaches 84.0% and 83.9% mIoU on *val* and *test* sets, both of which significantly outperform the original WeCLIP and other single-stage approaches by a large margin. Specifically, compared to WeCLIP, our WeCLIP+ brings 7.6% and 6.7% mIoU increase on *val* and *test* set, respectively. Besides, CPAL [41] is the previous state-of-the-art multi-stage approach, which is also a CLIP-based solution. Our approach performs much better than it, with 9.5% and 9.2% mIoU increase. More importantly, WeCLIP+ even outperforms S2C [43], which utilized a powerful pixel-level segmentation model SAM (ViT-H) [44] as the baseline to handle this task, and WeCLIP+ brings out 5.8% and 6.4% mIoU increase.

Table II shows the comparisons between our approach and previous state-of-the-art approaches on MS COCO-2014 *val* set. Our WeCLIP+ achieves new state-of-the-art performance, reaching 56.3% mIoU. Compared to other single-stage approaches, WeCLIP+ brings more than a 9.6% mIoU increase, which is a significant improvement. More importantly,

TABLE I
COMPARISON OF STATE-OF-THE-ART APPROACHES ON THE PASCAL VOC 2012 *VAL* AND *TEST* DATASET

Method	Backbone	Sup.	*val*	*test*
mutil-stage weakly supervised approaches				
$EPS_{CVPR'21}$ [70]	ResNet101	I+S	71.0	71.8
$RCA_{CVPR'22}$ [71]	ResNet101	I+S	72.2	72.8
$L2G_{CVPR'22}$ [10]	ResNet101	I+S	72.1	71.7
$Mat\text{-}label_{ICCV'23}$ [72]	ResNet101	I+S	73.3	74.0
$S\text{-}BCE_{ECCV'22}$ [23]	ResNet38	I+S	68.1	70.4
$RIB_{NeurIPS'21}$ [18]	ResNet38	I	68.3	68.6
$W\text{-}OoD_{CVPR'22}$ [73]	ResNet101	I	69.8	69.9
$ESOL_{NeurIPS'22}$ [25]	ResNet101	I	69.9	69.3
$VML_{IJCV'22}$ [74]	ResNet101	I	70.6	70.7
$AETF_{ECCV'22}$ [21]	ResNet38	I	70.9	71.7
$MCTformer_{CVPR'22}$ [9]	ViT+Res38	I	70.4	70.0
$CDL_{IJCV'23}$ [75]	ResNet101	I	72.4	72.2
$ACR_{CVPR'23}$ [76]	ViT-B	I	72.4	72.4
$BECO_{CVPR'23}$ [77]	MIT-B2	I	73.7	73.5
$FPR_{ICCV'23}$ [78]	ResNet101	I	70.0	70.6
$USAGE_{ICCV'23}$ [79]	ResNet38	I	71.9	72.8
$CLIMS_{CVPR'22}$ [24]	ViT+Res101	I+L	70.4	70.0
$CLIP\text{-}ES_{CVPR'23}$ [26]	ViT+Res101	I+L	73.8	73.9
$CTI_{CVPR'24}$ [80]	ViT+Res38	I	74.1	73.2
$PSDPM_{CVPR'24}$ [40]	ViT+Res101	I+L	74.1	74.9
$CPAL_{CVPR'24}$ [41]	ViT+Res101	I+L	74.5	74.7
$S2C_{CVPR'24}$ [43]	SAM [44]	I	**78.2**	**77.5**
single-stage weakly supervised approaches				
$1Stage_{CVPR'20}$ [13]	ResNet38	I	62.7	64.3
$RRM_{AAAI'20}$ [12]	ResNet38	I	62.6	62.9
$AA\&AR_{ACMMM'21}$ [16]	ResNet38	I	63.9	64.8
$SLRNet_{IJCV'22}$ [47]	ResNet38	I	67.2	67.6
$AFA_{CVPR'22}$ [14]	MIT-B1	I	66.0	66.3
$TSCD_{AAAI'23}$ [81]	MIT-B1	I	67.3	67.5
$ToCo_{CVPR'23}$ [48]	ViT-B	I	71.1	72.2
$DuPL_{CVPR'24}$ [50]	ViT-B	I	73.3	72.8
$SeCO_{CVPR'24}$ [51]	ViT-B	I	74.0	73.8
ours-WeCLIP (w/o CRF)	ViT-B	I+L	74.9	75.2
ours-WeCLIP (w/ CRF)	ViT-B	I+L	76.4	**77.2**
ours-WeCLIP+ (w/o CRF)	ViT-B-S_d†	I+L	80.7	81.0
ours-WeCLIP+ (w/ CRF)	ViT-B-S_d	I+L	82.4	82.8
ours-WeCLIP+ (w/o CRF)	ViT-B-B_d‡	I+L	82.4	82.2
ours-WeCLIP+ (w/ CRF)	ViT-B-B_d	I+L	83.5	83.4
ours-WeCLIP+ (w/ CRF)	ViT-B-L_d	I+L	**84.0**	**83.9**

mIoU (%) as the evaluation metric. I: image-level labels; S: saliency maps; L: language. mIoU as the evaluation metric. Without a specific description, results are reported with multi-scales and DenseCRF during inference.

† The CLIP encoder is ViT-B-16 and the DINO encoder is ViT-S-14.
‡ The CLIP encoder is ViT-B-16 and the DINO encoder is ViT-B-14.

TABLE II
COMPARISON WITH OTHER STATE-OF-THE-ART METHODS ON MS COCO-2014 *VAL* SET

Method	Backbone	Sup.	mIoU (%)
mutil-stage weakly supervised approaches			
$L2G_{CVPR'22}$ [10]	ResNet101	I+S	44.2
$RCA_{CVPR'22}$ [71]	ResNet101	I+S	36.8
$PMM_{ICCV'21}$ [17]	ResNet101	I	36.7
$RIB_{NeurIPS'21}$ [18]	ResNet101	I	43.8
$VWL_{IJCV'22}$ [74]	ResNet101	I	36.2
$MCTformer_{CVPR'22}$ [9]	ViT+Res38	I	42.0
$SIPE_{CVPR'22}$ [83]	ResNet38	I	43.6
$ESOL_{NeurIPS'22}$ [25]	ResNet101	I	42.6
$FPR_{ICCV'23}$ [78]	ResNet101	I	43.9
$USAGE_{ICCV'23}$ [79]	ResNet101	I	44.3
$CDL_{IJCV'23}$ [75]	ResNet101	I	**45.5**
$BECO_{CVPR'23}$ [77]	ResNet101	I	45.1
$CLIP\text{-}ES_{CVPR'23}$ [26]	ViT+Res101	I+L	45.4
$CTI_{CVPR'24}$ [80]	ViT+Res101	I	45.4
$PSDPM_{CVPR'24}$ [40]	ViT+Res101	I+L	47.2
$CPAL_{CVPR'24}$ [41]	ViT+Res101	I+L	46.8
$S2C_{CVPR'24}$ [43]	SAM [44]	I	**49.8**
single-stage weakly supervised approaches			
$SLRNet_{IJCV'22}$ [47]	ResNet38	I	35.0
$AFA_{CVPR'22}$ [14]	MIT-B1	I	38.9
$TSCD_{AAAI'23}$ [81]	MIT-B1	I	40.1
$ToCo_{CVPR'23}$ [48]	ViT-B	I	42.3
$DuPL_{CVPR'24}$ [50]	ViT-B	I	44.6
$SeCO_{CVPR'24}$ [51]	ViT-B	I	46.7
ours-WeCLIP (w/o CRF)	ViT-B	I+L	46.4
ours-WeCLIP (w/ CRF)	ViT-B	I+L	47.1
ours-WeCLIP (w/o CRF)	ViT-B-S_d	I+L	51.7
ours-WeCLIP (w/ CRF)	ViT-B-S_d	I+L	52.1
ours-WeCLIP (w/o CRF)	ViT-B-B_d	I+L	53.7
ours-WeCLIP (w/ CRF)	ViT-B-B_d	I+L	54.2
ours-WeCLIP (w/ CRF)	ViT-B-L_d	I+L	**56.3**

TABLE III
TRAINING COST COMPARISONS ON PASCAL VOC 2012 DATASET

Method	Train time	Maximum GPU memory
MCTformer [9]	25h	12G
CLIP-ES [26]	7h	12G
ToCo [48]	**4h**	>24G
WeCLIP	4.5h	**6.2G**
WeCLIP+ (ViT-B-S_d)	4.6h	7.2G
WeCLIP+ (ViT-B-B_d)	4.8h	7.6G

All methods are run on NVIDIA RTX 3090 GPUs.

TABLE IV
MODEL PARAMETERS COMPARISON WITH OTHER SINGLE-STAGE APPROACHES

Method	Train. #Params.	All #Params.	Flops	FPS
$ToCo_{(ViT\text{-}B)}$ [48]	98.9M	105.6M	93.8G	120.9
$DuPL_{(ViT\text{-}B)}$ [50]	184.7M	185.0M	187.6G	46.2
$SeCO_{(ViT\text{-}B)}$ [51]	95.0M	189.6M	88.3G	116.2
$WeCLIP_{(ViT\text{-}B)}$	5.7M	154.8M	105.9G	105.3
$WeCLIP+_{(ViT\text{-}B\text{-}S_d)}$	3.2M	173.2M	150.4G	74.1
$WeCLIP+_{(ViT\text{-}B\text{-}B_d)}$	3.2M	237.3M	250.0G	57.0
$WeCLIP+_{(ViT\text{-}B\text{-}L_d)}$	4.7M	456.4M	588.0G	28.8

The resolution is fixed as 512 × 512, and all experiments are conducted on the NVIDIA RTX 4090. "Train. #params" means trainable parameters.

WeCLIP+ also outperforms other multi-stage approaches by a clear margin with fewer training steps. Considering WeCLIP and WeCLIP+ use the frozen backbone, it shows great advantages to this task.

In Table III, we compare the training cost between our approach and other state-of-the-art approaches on the PASCAL VOC 2012 dataset. It can be seen that both WeCLIP and WeCLIP+ require small GPU memory, while other approaches require at least 12 G GPU memory. ToCo [48] has less training time than us, but its GPU memory is much higher than WeCLIP and WeCLIP+. More importantly, ToCo [48] spent 4 hours with 20,000 training iterations, while WeCLIP and WeCLIP+ spent approximately 5 h hours with 30,000 iterations, which also shows the high training efficiency of our approach.

Table IV compares the training and inference costs between our methods and other single-stage methods. Our approach has a better trade-off between efficiency and performance compared to other single-stage approaches. Notably, our approach features fewer learnable parameters than competing methods, ensuring that even our largest configuration, $WeCLIP+_{(ViT\text{-}B\text{-}L_d)}$, requires less than 12G GPU memory for both training and inference, i.e., just one NVIDIA 2080Ti GPU is enough. Besides, our

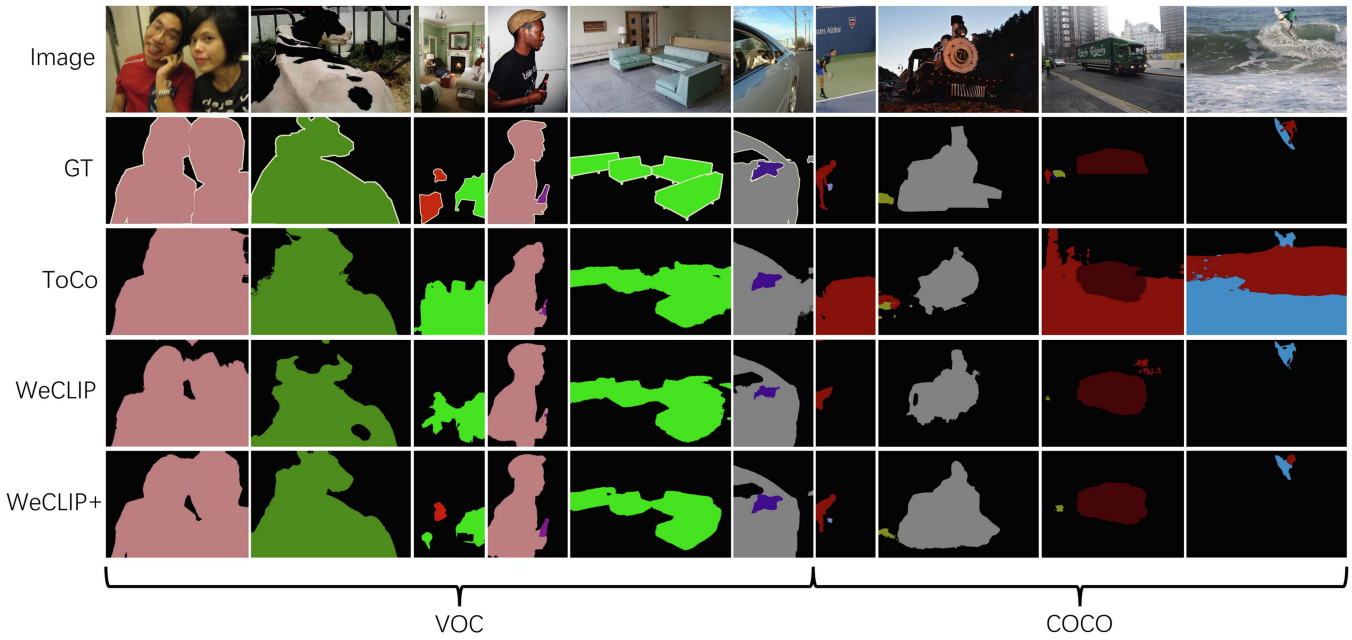


Fig. 6. Qualitative comparisons among WeCLIP+, WeCLIP and ToCo [48] on PASCAL VOC 2012 and MS COCO-2014 *val* set. Our approach generates more detailed visual results.

TABLE V
ABLATION STUDY OF EACH COMPONENT IN WECLIP AND WECLIP+ ON PASCAL VOC 2012 *VAL* SET

Method	Decoder	RFM	DINO	RFM+	mIoU(%)
WeCLIP	✓		-	-	68.7
	✓	✓	-	-	74.9
WeCLIP+	✓		✓		70.7
	✓	✓	✓		78.8
	✓		✓	✓	**82.4**

TABLE VI
ABLATION STUDY ABOUT TRANSFORMER LAYER NUMBERS IN ϕ OF (3) ON PASCAL VOC 2012 *VAL* SET WITH MIOU (%) AS THE EVALUATION METRIC

ϕ (Trans. Layer)	1	2	3	4	5
WeCLIP	73.2	74.4	**74.9**	72.6	70.3
WeCLIP+	81.9	82.2	**82.4**	81.9	81.7

approach still keeps a high inference speed, for example, The FPS of WeCLIP+$_{(\text{ViT-B-B}_d)}$ is 57.0, surpassing the 46.2 FPS of DuPL [50]. Such a good balance between performance and cost underscores the practicality and scalability of our methods.

In Fig. 6, we show some qualitative comparisons among our WeCLIP+, WeCLIP, and other approaches on the PASCAL VOC 2012 and MS COCO-2014 *val* set. The visual results show that our WeCLIP+ can generate more complete object details with accurate boundaries than our WeCLIP and ToCo [48] for both datasets.

D. Ablation Studies

We conduct ablation studies on the PASCAL VOC 2012 *val* set to evaluate the effectiveness of our approach. CRF is not used to refine the final prediction. Unless otherwise noted, for WeCLIP, the backbone is ViT-B. For WeCLIP+, ViT-B-B$_\text{d}$ is employed as the backbone, i.e., both the frozen CLIP backbone and the frozen DINO backbone are ViT-B.

Table V shows the influence of our each proposed component for WeCLIP and WeCLIP+. As a single-stage approach, the decoder is necessary. We cannot generate the prediction without it. For WeCLIP, introducing RFM brings a clear improvement, with a 6.2% mIoU increase. Since RFM is designed to improve the online pseudo labels, this gain also proves its effectiveness in generating higher quality pseudo labels. For WeCLIP+, directly introducing DINO in the backbone can bring a 3.9% mIoU increase with the original RFM. Besides, using RFM+ reaches 82.4% mIoU, which is much higher than using the original RFM, indicating RFM+ can produce higher quality than the original RFM.

Table VI reports the influence of the number of transformer layers in our decoder, i.e., ϕ in (3). The performance increases when the layer number increases to 3 for both WeCLIP and WeCLIP+. This is because the limited size of the decoder cannot capture enough feature information, and it is easy to under-fit the features. With the increase of layer number, the decoder learns better feature representation. However, the performance drops if the layer number is larger than 3. One possible reason is that deeper decoder layers cause over-fitting problem. Thus, it is reasonable that the performance drops after increasing to 4 or 5 for ϕ.

In Table VII, we conduct the ablation study to study the influence of different frozen image features selected as input for our decoder. Note here that we use N_0 to represent the selected starting transformer block number. For WeCLIP, when $N_0 = 1$, image features from all blocks in the frozen image encoder

TABLE VII
ABLATION STUDY OF THE INPUT FROZEN CLIP IMAGE FEATURES FOR DECODER ON PASCAL VOC 2012 *VAL* SET WITH MIOU (%) AS THE EVALUATION METRIC

$\{F^l_{\text{init}}\}^{l=12}_{l=N_0}$	1	5	8	11	12
WeCLIP	**74.9**	74.7	74.6	74.5	74.3
WeCLIP+	82.4	82.2	82.0	82.2	**82.4**

N_0 means the initial selected block number. "1, 5, 8, 11, 12" indicates the value of N_0. For example, $N_0 = 1$ means that frozen image features from 1 to 12 blocks (all blocks) are selected as input for the decoder. Note that for WeCLIP+, the DINO feature is fixed as the feature from the last block.

TABLE VIII
ABLATION STUDY OF THE DYNAMIC REFINING MAP R IN (7)

R			mIoU (%)
A_f	G_e	A_s	
✓			65.7
		✓	71.8
	✓	✓	72.3
✓		✓	74.3
✓	✓	✓	**74.9**

TABLE IX
ABLATION STUDY OF THE COMPONENTS IN (17) FOR WECLIP+

$\widetilde{R}_{\text{nor}}$	$\widetilde{P}$	mIoU(%)
✓		80.5
✓	✓	**82.4**

are selected, and the best performance is generated. Besides, increasing N_0 from 1 to 12 decreases the mIoU score from 74.9% to 74.3%, indicating that with fewer features selected, lower performance is generated. The possible reason is that using all features has a more comprehensive semantic representation. For WeCLIP+, when $N_0 = 12$, i.e., only the feature from the final block is selected, it generates the satisfied results, the same as selecting all the blocks, indicating that the introduced DINO model retains sufficient semantic information in the last block, thus achieving a stronger backbone than the original WeCLIP.

In Table VIII, we evaluate the effectiveness of our refining map generated in RFM. When only A_s is used, it means that all attention maps are selected to refine the CAM, i.e., the same process proposed in [26], it generates 71.8% mIoU score. Note that such a process is a static operation, which is not optimized during training. Introducing G_e and A_f clearly improves it with 0.5% and 2.5% mIoU increase, respectively. Finally, combining A_f, G_e, and A_s using (7) generates much better results than others, showing the effectiveness of our refining method. More importantly, using the affinity map from our decoder provides a dynamic refinement strategy, optimizing the refinement process during training.

For RFM+, Table IX conducts the ablation study about each component in (17). The newly introduced component $\widetilde{P}$ brings a 1.9% mIoU increase, which indicates that $\widetilde{P}$ can improve the quality of pseudo labels.

Table X is the ablation study for the different supervision signals of A_f. M_p means using the online pseudo labels for $\hat{A}$

TABLE X
ABLATION STUDY FOR THE SUPERVISION OF A_f WITH MIOU (%)

$\hat{A}$		WeCLIP	WeCLIP+*
M_p	$\text{argmax}(P)$		
✓		**74.9**	**82.4**
	✓	74.6	81.8
✓	✓	74.8	81.9

M_p is the online pseudo labels, P is the final prediction. Simultaneously using M_p and P means using the intersection between M_p and P.
* For WeCLIP+, M_p is $\widetilde{M}_p$, P is $\widetilde{P}$, $\hat{A}$ is $\hat{A}^+$.

TABLE XI
ABLATION STUDY OF THE HYPERPARAMETER λ IN (10) AND (18) FOR BALANCING THE LOSS FUNCTION USING MIOU (%)

λ	0	0.1	0.5	1.0
WeCLIP	73.3	**74.9**	73.6	72.9
WeCLIP+	79.5	**82.4**	76.2	75.3

TABLE XII
ABLATION STUDY OF THE HYPERPARAMETER β IN (19) USING MIOU (%) AS THE EVALUATION METRIC

β	0	0.1	0.5	1.0
WeCLIP+	81.3	**82.4**	80.2	76.9

and $\text{argmax}(P)$ means using the final prediction P for $\hat{A}$. The last row means using the intersection between M_p and P for $\hat{A}$. It can be found that when using the pseudo label M_p to produce $\hat{A}$ as supervision, it achieves 74.9% mIoU and 82.4% mIoU for WeCLIP and WeCLIP+, respectively. Both of them perform better than the other two cases. Using the prediction P cannot bring a higher mIoU score since P is updated during training, and it is easy to produce conflicting supervision, leading to an ineffective learning process.

Table XI shows the influence of the hyperparameter λ for balancing the loss function. When $\lambda = 0$, the learning of affinity map A_f is not supervised. It only generates a 73.3% mIoU for WeCLIP and 79.5% mIoU for WeCLIP+, respectively. This is because the uncontrolled A_f makes the filter G^l and refining map R unstable, thus reducing the quality of online pseudo labels. When $\lambda = 0.1$, it produces better results than others, showing a good balance between two loss functions.

In Table XII, we illustrate the influence of the hyperparameter β in (19). When $\beta = 0.1$, it performs better than other cases. Besides, when β increases from 0.1 to 1, the performance drops quickly, indicating that a large β will make the loss functions unbalanced, thus damaging the training process.

Table XIII shows the influence of the multi-scale strategy during inference. For WeCLIP, it can be seen that {0.75, 1.0} outperforms other settings. Introducing a larger scale, such as 1.5, does not improve the performance, showing that the Frozen CLIP backbone is not sensitive to the large scale. However, for WeCLIP+, using a large scale, e.g., {1.5}, produces better results than the small scale, which performs differently from WeCLIP.

Table XIV illustrates that a shared decoder makes better predictions than using two individual decoders. Besides, directly concatenating the CLIP and DINO features cannot achieve a

TABLE XIII
INFLUENCE OF DIFFERENT MULTI-SCALES DURING INFERENCE, WITH MIOU(%) AS THE EVALUATION METRIC

Multi-scale	WeCLIP	WeCLIP+
{1.0}	74.0	81.9
{0.5, 1.0}	74.2	80.5
{0.75, 1.0}	**74.9**	81.5
{0.5, 0.75, 1.0}	74.4	80.6
{0.75, 1.0, 1.25}	74.8	81.9
{0.75, 1.0, 1.5}	74.5	82.1
{1.0, 1.5}	74.2	**82.4**

TABLE XIV
ABLATION STUDY ABOUT THE DECODER ARCHITECTURE FOR WECLIP+

Architecture	mIoU(%)
$\phi(\text{cat}(F_{\text{clip}}, F_{\text{dino}}))$	81.0
$\phi_1(F_{\text{clip}}) + \phi_2(F_{\text{dino}})$	80.5
$\phi(F_{\text{clip}}) + \phi(F_{\text{dino}})$	**82.4**

"$\phi(\cdot)$" means the module in Eq. (13). "cat" means two features are concatenated. "$\phi_1(\cdot)$ and $\phi_2(\cdot)$" means using two individual modules.

TABLE XV
ABLATION STUDY ABOUT THE PREDICTIONS OF CLIP BRANCH, DINO BRANCH AND THE FINAL COMBINATION OF THE CLIP AND DINO PREDICTIONS FOR WECLIP+

Prediction	P_{clip}	P_{dino}	$\widetilde{P}$	only F_{clip}	only F_{dino}
mIoU (%)	77.2	81.3	**82.4**	73.2	79.3

"only F_{clip}" means only F_{clip} is used as input for the decoder, and "only F_{dino}" means only F_{dino} is used for the decoder.

performance similar to that of the shared decoder. One possible reason is that for weakly supervised semantic segmentation, the pseudo labels contain some noisy annotations, with the shared decoder, WeCLIP+ can provide cross-supervisions for two branches to achieve a robust learning process. Moreover, the DINO feature contains sufficient semantic information, once concatenating two features, the mixed features will contain information in CLIP that is related to the image-level representation, compromising the performance.

In Table XV, we make a comparison of the final prediction. It can be seen that P_{dino} is higher than P_{clip}, which illustrates that the DINO feature is suitable for semantic segmentation. Finally, $\widetilde{P}$ generates the best performance, which proves that the final prediction benefits from both the CLIP feature and the DINO feature. Besides, we also evaluate the performance when only input F_{clip} or F_{dino} to the decoder, only with F_{clip}, the performance is 73.2%, less than the original WeCLIP (74.9%), as F_{clip} only contains the feature map from last layer of the last block in CLIP. Therefore, when using only CLIP features in the decoder, multiple features from different layers are necessary for generating better segmentation results. When only using F_{dino}, the performance is 79.3%, showing the advantages of DINO features to tackle the dense prediction task. More importantly, compare the performance between "P_{dino}" and "only F_{dino}", combing CLIP and DINO features for decoding brings 2.0% mIoU increase, which illustrates the frozen DINO feature benefits from the shared decoder architecture.

In Fig. 8, we further report the refined pseudo label quality after RFM and RFM+ in different training iterations for both WeCLIP and WeCLIP+. It is observed that the quality of pseudo labels improves with training iterations for both WeCLIP and WeCLIP+. The quality of pseudo labels generated by WeCLIP+ is higher than WeCLIP for all training iterations. Such a phenomenon verifies that RFM+ makes better use of the combination of the frozen CLIP and DINO features to refine the initial CAMs.

Fig. 9 shows the final performance implications of using different backbones and using different quality pseudo labels. We design a two-stage pipeline to evaluate it on the PASCAL VOC 2012 dataset, i.e., we first use some approaches to generate different quality pseudo labels of the training set and then use such labels to directly train our fully supervised WeCLIP and WeCLIP+ (Fig. 7). For generating different quality pseudo labels, we use ES-CLIP [26] (75.0%), WeCLIP (78.1%), and WeCLIP+ (with different backbones, 83.8%, 84.9% and 85.0%). Then, we train our fully supervised framework with such pseudo labels. On the one hand, it can be found stronger backbone representations bring performance increases by less than 4%, e.g., replacing WeCLIP+$_{(\text{ViT-B-S}_d)}$ with WeCLIP+$_{(\text{ViT-B-L}_d)}$, the gain is less than 2% across all-level pseudo labels. On the other hand, high quality pseudo-labels can bring significant final performance improvement. For example, the performance of WeCLIP and WeCLIP+ have increased approximately 6% mIoU when the quality of the pseudo label is improved from 75.0% to 85.0%. In summary, under this framework, pseudo-label quality is the main contributing factor to the final segmentation performance, while a stronger backbone is a secondary factor.

E. Performance on Fully-Supervised Semantic Segmentation

We also use our WeCLIP and WeCLIP+ to tackle fully-supervised semantic segmentation. The frameworks of WeCLIP and WeCLIP+ for full supervision are demonstrated in Fig. 7. For fully-supervised semantic segmentation, it provides accurate pixel-level labels, so we remove the frozen text encoder and our RFM/RFM+, only keeping the frozen image encoder and our decoder. Besides, the loss function only remains the cross-entropy loss between the final predictions and the ground-truth.

In Table XVI, we evaluate our approach on the PASCAL VOC 2012 set for fully supervised semantic segmentation. Since our approach utilizes a frozen backbone, it has less trainable parameters, but high-level segmentation performance is maintained, showing its great potential for fully-supervised semantic segmentation. Specifically, WeCLIP+ decreases the number of learnable parameters to 58% of WeCLIP and 24% of the previous best method, SegNeXt-S [85]. More importantly, for weakly supervised semantic segmentation, the difference is just adding the pseudo-label generation part, which has no learnable parameters. Thus, our WeCLIP+ can build a stronger single-stage pipeline

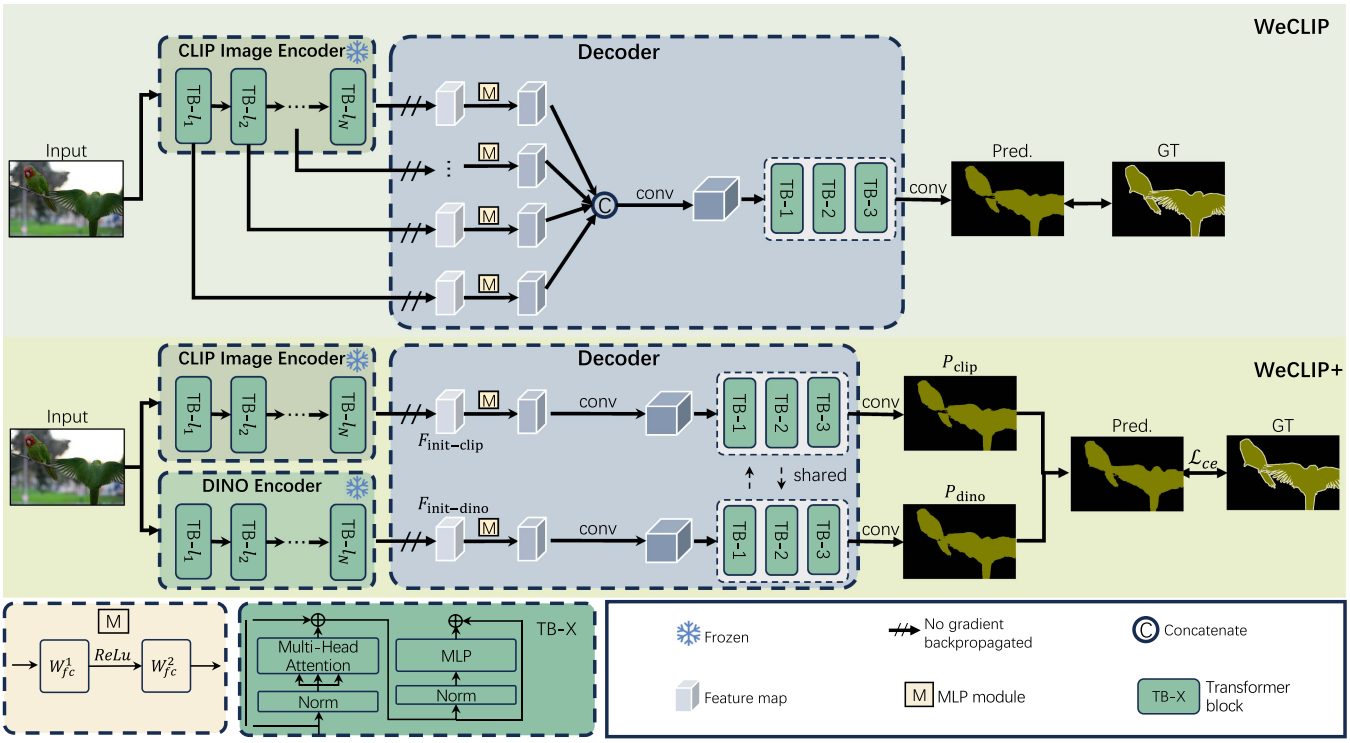


Fig. 7. The framework of our WeCLIP and WeCLIP+ for fully-supervised semantic segmentation. For this task, both our WeCLIP and WeCLIP+ remove the settings for the text encoder and the RFM (RFM+). For WeCLIP, the input image passes the frozen CLIP image encoder to extract the feature maps. Then, the feature maps of the last layers of each transformer block are input to our decoder to generate the final prediction. On the other hand, for WeCLIP+, the image is input to the Frozen CLIP image encoder and Frozen DINO to generate two individual image feature maps, $F_{\text{init-clip}}$ and $F_{\text{init-dino}}$. Then, the two feature maps are input to the parameter-shared decoder for the two predictions, P_{clip} and P_{dino}, respectively. Finally, the combination of the two predictions is treated as the final output. We use the ground-truth to supervise WeCLIP and WeCLIP+ directly with the cross-entropy loss.

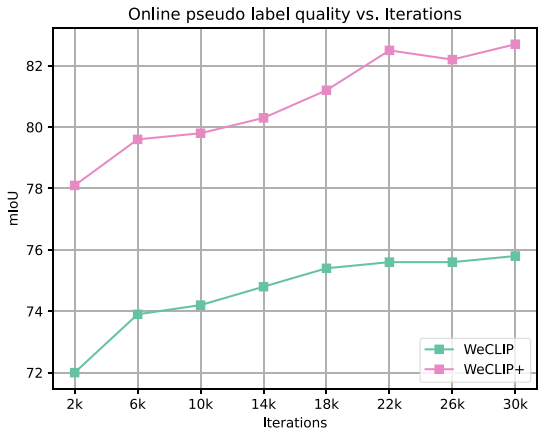


Fig. 8. Quality of online pseudo label vs. Iterations. It can be found that our WeCLIP+ does generate higher quality online pseudo labels. Such pseudo labels are better than those generated by WeCLIP from the beginning iterations.

TABLE XVI
PERFORMANCE ON PASCAL VOC 2012 *VAL* AND *TEST* SET FOR FULLY-SUPERVISED SEMANTIC SEGMENTATION

Method	Backbone	L. Params	*val*	*test*
DeepLabV3*	ResNet101	58M	79.9	79.8
Mask2former [62]*	ResNet50	44M	77.3	-
SegNeXt-S [85]	MSCAN-S	13.9M	-	**85.3**
WeCLIP	ViT-B	5.7M	81.6	81.1
WeCLIP+	ViT-B-S_d	3.2M	85.7	85.8
WeCLIP+	ViT-B-B_d	3.2M	**87.0**	**87.2**

mIoU is used as the evaluation metric. "L. Params" means learnable parameters during training. * results are reproduced by [86].

that has fewer learnable parameters but higher performance with the introduced DINO model in the backbone.

To study why the vision feature from the frozen CLIP and the Frozen DINO can be directly used for semantic segmentation, we show feature visualization results for the CLIP features, the DINO features, and ImageNet features in Fig. 10. We randomly select 200 images from the PASCAL VOC 2012 *train* set. Without any training or finetuning, we use ViT-B as the backbone and directly initialize it with frozen pre-train weights. It can be found that features belonging to the same class, pre-trained by CLIP and DINO, are denser and clustered but with different distributions. Conversely, features belonging to the same class, pre-trained by ImageNet, are sparse and decentralized. Fig. 10 indicates that the extracted features from the CLIP model and the DINO can better represent semantic information for different classes. Then, combining CLIP and DINO makes features evidently represent each class. Such discriminative features are more suitable for conducting segmentation tasks.

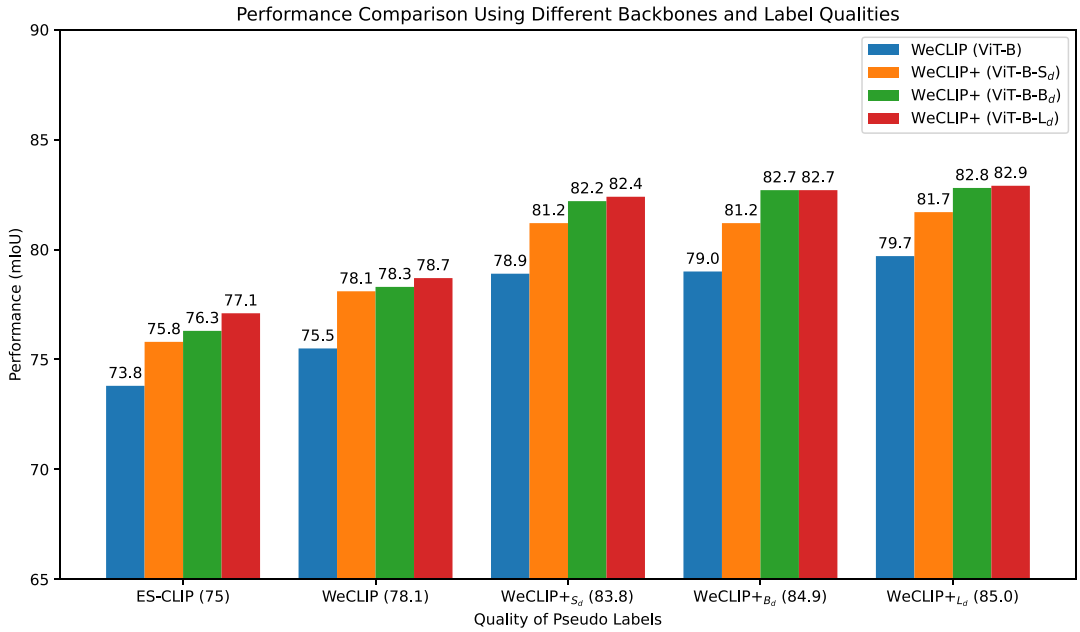


Fig. 9. Performance comparisons using different backbones under different quality of pseudo labels on PASCAL VOC *val* set. In the x-axis, "75.0" is generated using ES-CLIP [26], "78.1" is generated using our trained WeCLIP, "83.8" is generated using our trained WeCLIP+$_{(\text{ViT-B-S}_d)}$. "84.9" is generated using our trained WeCLIP+$_{(\text{ViT-B-B}_d)}$. "85.0" is generated using our trained WeCLIP+$_{(\text{ViT-B-L}_d)}$. The quality of pseudo labels is evaluated on the PASCAL VOC *train* set.

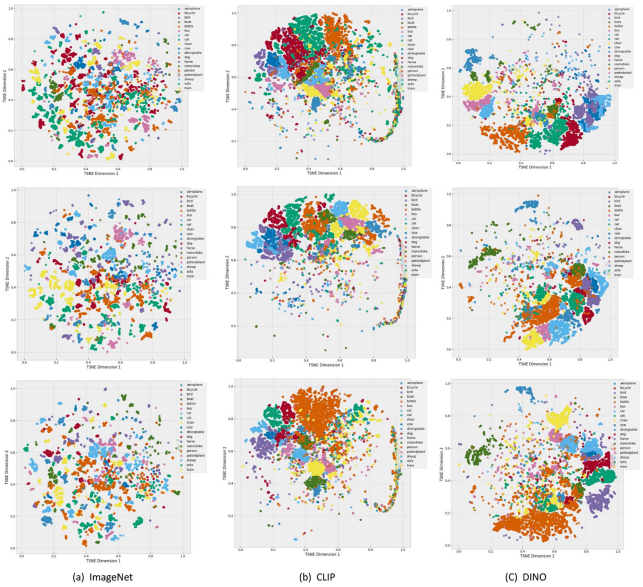


Fig. 10. Feature visualization with T-SNE [84] to show why frozen CLIP and frozen DINO can be used for semantic segmentation. Each color represents one specific category. (a) Frozen ImageNet pre-trained feature visualization of ViT-B. (b) Frozen CLIP pre-trained feature visualization of VIT-B. (c) Frozen DINO pre-trained feature visualization of VIT-B. It can be seen that without any retraining, the features belonging to the same class from the frozen CLIP and DINO are more compact compared with that in (a). Best viewed in color.

V. Conclusion

We design single-stage pipelines WeCLIP and WeCLIP+ to directly use the frozen CLIP and DINO model to extract semantic segmentation for weakly supervised semantic segmentation. For WeCLIP, we design a light decoder and a frozen CLIP CAM refinement module. Based on it, for WeCLIP+, we design a frozen CLIP-DINO feature decoder to interpret the features from the last blocks of the backbone and thus have fewer learnable parameters. Meanwhile, we propose a frozen CLIP-DINO CAM refinement module, which uses the learnable feature relationship from our decoder to refine CAM so as to improve the quality of pseudo labels. Our approach achieves better performance with less training cost, showing great advantages to tackle this task. We also evaluate the effectiveness of our approach to fully-supervised semantic segmentation. Our solution offers a different perspective from traditional approaches, which is that training the backbone is unnecessary. We believe the proposed approach can further boost research in this direction.

References

[1] J. Ahn, S. Cho, and S. Kwak, "Weakly supervised learning of instance segmentation with inter-pixel relations," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 2209–2218.

[2] Y. Wei et al., "Learning to segment with image-level annotations," *Pattern Recognit.*, vol. 59, pp. 234–244, 2016.

[3] B. Zhang, J. Xiao, J. Jiao, Y. Wei, and Y. Zhao, "Affinity attention graph neural network for weakly supervised semantic segmentation," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 11, pp. 8082–8096, Nov. 2022.

[4] Z. Liang, T. Wang, X. Zhang, J. Sun, and J. Shen, "Tree energy loss: Towards sparsely annotated semantic segmentation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 16907–16916.

[5] C. Song, Y. Huang, W. Ouyang, and L. Wang, "Box-driven class-wise region masking and filling rate guided loss for weakly supervised semantic segmentation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 3136–3145.

[6] A. Bearman, O. Russakovsky, V. Ferrari, and L. Fei-Fei, "What's the point: Semantic segmentation with point supervision," in *Proc. Eur. Conf. Comput. Vis.*, 2016, pp. 549–565.

[7] J. Ahn and S. Kwak, "Learning pixel-level semantic affinity with image-level supervision for weakly supervised semantic segmentation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 4981–4990.

[8] Y. Wang, J. Zhang, M. Kan, S. Shan, and X. Chen, "Self-supervised equivariant attention mechanism for weakly supervised semantic segmentation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 12275–12284.

[9] L. Xu, W. Ouyang, M. Bennamoun, F. Boussaid, and D. Xu, "Multi-class token transformer for weakly supervised semantic segmentation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 4310–4319.

[10] P.-T. Jiang, Y. Yang, Q. Hou, and Y. Wei, "L2G: A simple local-to-global knowledge transfer framework for weakly supervised semantic segmentation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 16886–16896.

[11] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2009, pp. 248–255.

[12] B. Zhang, J. Xiao, Y. Wei, M. Sun, and K. Huang, "Reliability does matter: An end-to-end weakly supervised semantic segmentation approach," in *Proc. Conf. Assoc. Adv. Artif. Intell.*, 2020, pp. 12765–12772.

[13] N. Araslanov and S. Roth, "Single-stage semantic segmentation from image labels," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 4253–4262.

[14] L. Ru, Y. Zhan, B. Yu, and B. Du, "Learning affinity from attention: End-to-end weakly-supervised semantic segmentation with transformers," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 16846–16855.

[15] B. Zhang, J. Xiao, Y. Wei, K. Huang, S. Luo, and Y. Zhao, "End-to-end weakly supervised semantic segmentation with reliable region mining," *Pattern Recognit.*, vol. 128, 2022, Art. no. 108663.

[16] X. Zhang et al., "Adaptive affinity loss and erroneous pseudo-label refinement for weakly supervised semantic segmentation," in *Proc. 29th ACM Int. Conf. Multimedia*, 2021, pp. 5463–5472.

[17] Y. Li, Z. Kuang, L. Liu, Y. Chen, and W. Zhang, "Pseudo-mask matters in weakly-supervised semantic segmentation," in *Proc. Int. Conf. Comput. Vis.*, 2021, pp. 6964–6973.

[18] J. Lee, J. Choi, J. Mok, and S. Yoon, "Reducing information bottleneck for weakly supervised semantic segmentation," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2021, pp. 27408–27421.

[19] B. Zhou, A. Khosla, A. Lapedriza, A. Oliva, and A. Torralba, "Learning deep features for discriminative localization," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 2921–2929.

[20] F. Zhang, C. Gu, C. Zhang, and Y. Dai, "Complementary patch for weakly supervised semantic segmentation," in *Proc. Int. Conf. Comput. Vis.*, 2021, pp. 7242–7251.

[21] S.-H. Yoon, H. Kweon, J. Cho, S. Kim, and K.-J. Yoon, "Adversarial erasing framework via triplet with gated pyramid pooling layer for weakly supervised semantic segmentation," in *Proc. Eur. Conf. Comput. Vis.*, 2022, pp. 326–344.

[22] Y. Du, Z. Fu, Q. Liu, and Y. Wang, "Weakly supervised semantic segmentation by pixel-to-prototype contrast," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 4320–4329.

[23] T. Wu, G. Gao, J. Huang, X. Wei, X. Wei, and C. H. Liu, "Adaptive spatial-BCE loss for weakly supervised semantic segmentation," in *Proc. Eur. Conf. Comput. Vis.*, 2022, pp. 199–216.

[24] J. Xie, X. Hou, K. Ye, and L. Shen, "CLIMS: Cross language image matching for weakly supervised semantic segmentation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 4483–4492.

[25] J. Li, Z. Jie, X. Wang, X. Wei, and L. Ma, "Expansion and shrinkage of localization for weakly-supervised semantic segmentation," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2022, pp. 16037–16051.

[26] Y. Lin et al., "Clip is also an efficient segmenter: A text-driven approach for weakly supervised semantic segmentation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 15305–15314.

[27] A. Radford et al., "Learning transferable visual models from natural language supervision," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 8748–8763.

[28] H. Zhu, Y. Wei, X. Liang, C. Zhang, and Y. Zhao, "CTP: Towards vision-language continual pretraining via compatible momentum contrast and topology preservation," in *Proc. Int. Conf. Comput. Vis.*, 2023, pp. 22257–22267.

[29] G. Zhang, L. Wang, G. Kang, L. Chen, and Y. Wei, "SLCA: Slow learner with classifier alignment for continual learning on a pre-trained model," in *Proc. Int. Conf. Comput. Vis.*, 2023, pp. 19148–19158.

[30] T. Huang et al., "CLIP2point: Transfer clip to point cloud classification with image-depth pre-training," in *Proc. Int. Conf. Comput. Vis.*, 2023, pp. 22157–22167.

[31] S. Jiao, Y. Wei, Y. Wang, Y. Zhao, and H. Shi, "Learning mask-aware clip representations for zero-shot segmentation," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2023, pp. 35631–35653.

[32] B. Zhang, S. Yu, Y. Wei, Y. Zhao, and J. Xiao, "Frozen clip: A strong backbone for weakly supervised semantic segmentation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2024, pp. 3796–3806.

[33] M. Oquab et al., "DINOv2: Learning robust visual features without supervision," 2023, *arXiv:2304.07193*.

[34] S. Kim, D. Park, and B. Shim, "Semantic-aware superpixel for weakly supervised semantic segmentation," in *Proc. Conf. Assoc. Adv. Artif. Intell.*, 2023, pp. 1142–1150.

[35] C. Ma, Y. Yang, Y. Wang, Y. Zhang, and W. Xie, "Open-vocabulary semantic segmentation with frozen vision-language models," 2022, *arXiv:2210.15138*.

[36] M. Wysoczańska, O. Siméoni, M. Ramamonjisoa, A. Bursuc, T. Trzciński, and P. Pérez, "CLIP-DINOiser: Teaching CLIP a few DINO tricks," 2023, *arXiv:2312.12359*.

[37] Z. Chen, T. Wang, X. Wu, X.-S. Hua, H. Zhang, and Q. Sun, "Class re-activation maps for weakly-supervised semantic segmentation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 969–978.

[38] J. He, L. Cheng, C. Fang, D. Zhang, Z. Wang, and W. Chen, "Mitigating undisciplined over-smoothing in transformer for weakly supervised semantic segmentation," 2023, *arXiv:2305.03112*.

[39] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-CAM: Visual explanations from deep networks via gradient-based localization," in *Proc. Int. Conf. Comput. Vis.*, 2017, pp. 618–626.

[40] X. Zhao, Z. Yang, T. Dai, B. Zhang, and J. Xiao, "PSDPM: Prototype-based secondary discriminative pixels mining for weakly supervised semantic segmentation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2024, pp. 3437–3446.

[41] F. Tang, Z. Xu, Z. Qu, W. Feng, X. Jiang, and Z. Ge, "Hunting attributes: Context prototype-aware learning for weakly supervised semantic segmentation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2024, pp. 3324–3334.

[42] L. Zhu et al., "WeakCLIP: Adapting CLIP for weakly-supervised semantic segmentation," *Int. J. Comput. Vis.*, vol. 133, pp. 1085–1105, 2024.

[43] H. Kweon and K.-J. Yoon, "From sam to cams: Exploring segment anything model for weakly supervised semantic segmentation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2024, pp. 19499–19509.

[44] A. Kirillov et al., "Segment anything," in *Proc. Int. Conf. Comput. Vis.*, 2023, pp. 4015–4026.

[45] Z. Cheng et al., "Out-of-candidate rectification for weakly supervised semantic segmentation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 23673–23684.

[46] D. Zhang et al., "Weakly supervised semantic segmentation via alternate self-dual teaching," *IEEE Trans. Image Process.*, 2023, doi: 10.1109/TIP.2023.3343112.

[47] J. Pan et al., "Learning self-supervised low-rank network for single-stage weakly and semi-supervised semantic segmentation," *Int. J. Comput. Vis.*, vol. 130, no. 5, pp. 1181–1195, 2022.

[48] L. Ru, H. Zheng, Y. Zhan, and B. Du, "Token contrast for weakly-supervised semantic segmentation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 3093–3102.

[49] J. He, L. Cheng, C. Fang, Z. Feng, T. Mu, and M. Song, "Progressive feature self-reinforcement for weakly supervised semantic segmentation," in *Proc. Conf. Assoc. Adv. Artif. Intell.*, 2024, pp. 2085–2093.

[50] Y. Wu, X. Ye, K. Yang, J. Li, and X. Li, "Dupl: Dual student with trustworthy progressive learning for robust weakly supervised semantic segmentation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2024, pp. 3534–3543.

[51] Z. Yang, K. Fu, M. Duan, L. Qu, S. Wang, and Z. Song, "Separate and conquer: Decoupling co-occurrence via decomposition and representation for weakly supervised semantic segmentation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2024, pp. 3606–3615.

[52] J. Long, E. Shelhamer, and T. Darrell, "Fully convolutional networks for semantic segmentation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2015, pp. 3431–3440.

[53] L.-C. Chen, G. Papandreou, I. Kokkinos, K. Murphy, and Yuille, "DeepLab: Semantic image segmentation with deep convolutional nets, atrous convolution, and fully connected CRFs," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 40, no. 4, pp. 834–848, Apr. 2018.

[54] H. Zhao, J. Shi, X. Qi, X. Wang, and J. Jia, "Pyramid scene parsing network," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2017.

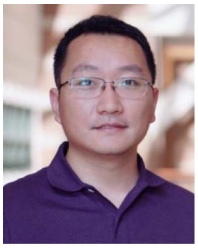
[55] T. Xiao, Y. Liu, B. Zhou, Y. Jiang, and J. Sun, "Unified perceptual parsing for scene understanding," in *Proc. Eur. Conf. Comput. Vis.*, 2018, pp. 418–434.

[56] Z. Jin, X. Hu, L. Zhu, L. Song, L. Yuan, and L. Yu, "IDRNet: Intervention-driven relation network for semantic segmentation," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2024, pp. 51606–51620.
[57] C. Sung, W. Kim, J. An, W. Lee, H. Lim, and H. Myung, "Contextrast: Contextual contrastive learning for semantic segmentation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2024, pp. 3732–3742.
[58] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, and N. Houlsby, "An image is worth 16x16 words: Transformers for image recognition at scale," in *Proc. Int. Conf. Learn. Representations*, 2021, pp. 1–21.
[59] W. Wang et al., "Pyramid vision transformer: A versatile backbone for dense prediction without convolutions," 2021, *arXiv:2102.12122*.
[60] Z. Liu et al., "Swin transformer: Hierarchical vision transformer using shifted windows," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 10012–10022.
[61] B. Cheng, A. Schwing, and A. Kirillov, "Per-pixel classification is not all you need for semantic segmentation," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2021, pp. 17864–17875.
[62] B. Cheng, I. Misra, A. G. Schwing, A. Kirillov, and R. Girdhar, "Masked-attention mask transformer for universal image segmentation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 1290–1299.
[63] R. Sinkhorn, "A relationship between arbitrary positive matrices and doubly stochastic matrices," *Ann. Math. Statist.*, vol. 35, no. 2, pp. 876–879, 1964.
[64] F. Milletari, N. Navab, and S.-A. Ahmadi, "V-Net: Fully convolutional neural networks for volumetric medical image segmentation," in *Proc. 4th Int. Conf. 3D Vis.*, 2016, pp. 565–571.
[65] M. Everingham, L. Van Gool, C. K. Williams, J. Winn, and A. Zisserman, "The PASCAL visual object classes (VOC) challenge," *Int. J. Comput. Vis.*, vol. 88, no. 2, pp. 303–338, 2010.
[66] T.-Y. Lin et al., "Microsoft COCO: Common objects in context," in *Proc. Eur. Conf. Comput. Vis.*, 2014, pp. 740–755.
[67] B. Hariharan, P. Arbeláez, L. Bourdev, S. Maji, and J. Malik, "Semantic contours from inverse detectors," in *Proc. Int. Conf. Comput. Vis.*, 2011, pp. 991–998.
[68] A. Dosovitskiy et al., "An image is worth 16x16 words: Transformers for image recognition at scale," in *Proc. Int. Conf. Learn. Representations*, 2021, pp. 1–22.
[69] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Proc. Int. Conf. Learn. Representations*, 2017, pp. 1–8.
[70] S. Lee, M. Lee, J. Lee, and H. Shim, "Railroad is not a train: Saliency as pseudo-pixel supervision for weakly supervised semantic segmentation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 5495–5505.
[71] T. Zhou, M. Zhang, F. Zhao, and J. Li, "Regional semantic contrast and aggregation for weakly supervised semantic segmentation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 4299–4309.
[72] C. Wang, R. Xu, S. Xu, W. Meng, and X. Zhang, "Treating pseudo-labels generation as image matting for weakly supervised semantic segmentation," in *Proc. Int. Conf. Comput. Vis.*, 2023, pp. 755–765.
[73] J. Lee, S. J. Oh, S. Yun, J. Choe, E. Kim, and S. Yoon, "Weakly supervised semantic segmentation using out-of-distribution data," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 16897–16906.
[74] L. Ru, B. Du, Y. Zhan, and C. Wu, "Weakly-supervised semantic segmentation with visual words learning and hybrid pooling," *Int. J. Comput. Vis.*, vol. 130, no. 4, pp. 1127–1144, 2022.
[75] B. Zhang, J. Xiao, Y. Wei, and Y. Zhao, "Credible dual-expert learning for weakly supervised semantic segmentation," *Int. J. Comput. Vis.*, vol. 131, pp. 1892–1908, 2023.
[76] H. Kweon, S.-H. Yoon, and K.-J. Yoon, "Weakly supervised semantic segmentation via adversarial learning of classifier and reconstructor," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 11329–11339.
[77] S. Rong, B. Tu, Z. Wang, and J. Li, "Boundary-enhanced co-training for weakly supervised semantic segmentation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 19574–19584.
[78] L. Chen, C. Lei, R. Li, S. Li, Z. Zhang, and L. Zhang, "FPR: False positive rectification for weakly supervised semantic segmentation," in *Proc. Int. Conf. Comput. Vis.*, 2023, pp. 1108–1118.
[79] Z. Peng, G. Wang, L. Xie, D. Jiang, W. Shen, and Q. Tian, "USAGE: A unified seed area generation paradigm for weakly supervised semantic segmentation," in *Proc. Int. Conf. Comput. Vis.*, 2023, pp. 624–634.
[80] S.-H. Yoon, H. Kwon, H. Kim, and K.-J. Yoon, "Class tokens infusion for weakly supervised semantic segmentation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2024, pp. 3595–3605.
[81] R. Xu, C. Wang, J. Sun, S. Xu, W. Meng, and X. Zhang, "Self correspondence distillation for end-to-end weakly-supervised semantic segmentation," in *Proc. Conf. Assoc. Adv. Artif. Intell.*, 2023, pp. 3045–3053.
[82] P. Krähenbühl and V. Koltun, "Parameter learning and convergent inference for dense random fields," in *Proc. Int. Conf. Mach. Learn.*, 2013, pp. 513–521.
[83] Q. Chen, L. Yang, J.-H. Lai, and X. Xie, "Self-supervised image-specific prototype exploration for weakly supervised semantic segmentation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 4288–4298.
[84] L. Van der Maaten and G. Hinton, "Visualizing data using t-SNE," *J. Mach. Learn. Res.*, vol. 9, no. 11, pp. 2579–2605, 2008.
[85] M.-H. Guo, C.-Z. Lu, Q. Hou, Z. Liu, M.-M. Cheng, and S.-M. Hu, "Segnext: Rethinking convolutional attention design for semantic segmentation," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2022, pp. 1140–1156.
[86] Q. Nguyen, T. Vu, A. Tran, and K. Nguyen, "Dataset diffusion: Diffusion-based synthetic dataset generation for pixel-level semantic segmentation," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2023, pp. 76872–76892.

Bingfeng Zhang (Member, IEEE) received the BS degree in electronic information engineering from China University of Petroleum (East China), Qingdao, China, in 2015, the ME degree in systems, control, and signal processing from the University of Southampton, Southampton, U.K., in 2016, and the PhD degree from the University of Liverpool, Liverpool, U.K. He is an associate professor in the School of Control Science and Engineering with the China University of Petroleum (East China), Qingdao, China. His current research interest is computer vision, including semantic segmentation with limited annotation, salient object detection, and video object segmentation.

Siyue Yu (Member, IEEE) recieved BS degree from Xi'an Jiaotong-Liverpool University (Department of Electrical and Electronic Engineering), in 2022 and the PhD degree from the University of Liverpool (School of Electrical and Electronic Engineering). Joined Xi'an Jiaotong-Liverpool University, in 2023 as an assistant professor. Her current research interests include computer vision, including weakly supervised semantic segmentation, salient object detection, and anomaly detection.

Jimin Xiao (Member, IEEE) received the BS and ME degrees in telecommunication engineering from the Nanjing University of Posts and Telecommunications, Nanjing, China, in 2004 and 2007, respectively, and the PhD degree in electrical engineering and electronics from the University of Liverpool, Liverpool, in 2013. From 2013 to 2014, he was a senior researcher with the Department of Signal Processing, Tampere University of Technology, Tampere, Finland, and an External Researcher with the Nokia Research Center, Tampere. He is a Professor in the Department of Intelligent Science at Xi'an Jiaotong-Liverpool University, Suzhou, China. His research interests include image and video processing, computer vision, and deep learning. He has published more than 100 papers in top journals and conferences, including *IEEE Transactions on Pattern Analysis and Machine Intelligence*, *International Journal of Computer Vision*, *IEEE Transactions on Image Processing*, CVPR, ICCV, ECCV, AAAI, IJCAI, etc.



Yunchao Wei (Member, IEEE) is currently a full professor of Beijing Jiaotong University. He was selected as MIT TR35 China by MIT Technology Review, in 2021 and was named as one of the five top early-career researchers in Engineering and Computer Sciences in Australia by The Australian in 2020. He received the Discovery Early Career Researcher Award of the Australian Research Council, in 2019, the 1st Prize in Science and Technology awarded by China Society of Image and Graphics (CSIG), in 2019. He has published more than 100 papers in top-tier conferences/journals, Google citations 14000+. He received many competition prizes from CVPR/ICCV/ECCV, such as the Winner prizes of ILSVRC 2014, LIP 2018/2019, Youtube VOS 2021, Runner-up Prizes of ILSVRC 2017, DAVIS 2020, etc. He organized many workshops on top-tier conferences, including Learning from Imperfect Data Workshop series (CVPR 2019, 2020, 2021) and Real-world Recognition from Low-quality Inputs Workshop series (ICCV 2019, ECCV 2020). He has broad research interests in computer vision and machine learning. His current research interest focuses on visual recognition with imperfect data, image/video segmentation and object detection, and multi-modal perception.

Yao Zhao (Fellow, IEEE) received the BS degree from the Radio Engineering Department, Fuzhou University, Fuzhou, China, in 1989, the ME degree from the Radio Engineering Department, Southeast University, Nanjing, China, in 1992, and the PhD degree from the Institute of Information Science, Beijing Jiaotong University (BJTU), Beijing, China, in 1996, where he became an associate professor and a professor, in 1998 and 2001, respectively. From 2001 to 2002, he was a senior research fellow with the Information and Communication Theory Group, Faculty of Information Technology and Systems, Delft University of Technology, Delft, The Netherlands. In 2015, he visited the Swiss Federal Institute of Technology, Lausanne (EPFL), Switzerland. From 2017 to 2018, he visited University of Southern California. He is currently the Director with the Institute of Information Science, BJTU. His current research interests include image/video coding, digital watermarking and forensics, video analysis and understanding, and artificial intelligence. Dr. Zhao is a the IET. He serves on the editorial boards of several international journals, including as an associate editor for *IEEE Transactions on Cybernetics*, a senior associate editor for the *IEEE Signal Processing Letters*, and an area editor for Signal Processing: Image Communication. He was named a Distinguished Young Scholar by the National Science Foundation of China, in 2010 and was elected as a Chang Jiang Scholar of Ministry of Education of China, in 2013.